Development of a Method for External Penetration Testing of a Web Application for Vulnerabilities

By

Mavlanov Shakhrukh

School of Cybersecurity
Astana IT University

6B06301 — Cybersecurity
Supervisor: PhD Amirova Akzhibek

June 2026
Astana

Abstract

This diploma project presents the design, implementation, and experimental evaluation of **ReconX**, an external penetration testing framework for web applications. The proposed framework follows a black-box methodology and combines passive reconnaissance, active fingerprinting, configuration analysis, and a wide spectrum of vulnerability probes against a remote target whose internal source code and credentials are not available to the tester.

The framework is implemented in **Python 3** and is organized as a pipeline of 46 specialized modules grouped into 15 execution stages. Reconnaissance modules gather subdomain, DNS, port, and technology information; configuration modules audit **TLS**, security headers, **CORS**, authentication cookies, and **JWT** tokens; active probes test for the most relevant **OWASP Top 10** classes, including *reflected XSS*, *SQL injection*, *IDOR*, *host header injection*, *cache poisoning*, *prototype pollution*, *open redirect*, *SSRF/SSTI*, *XXE*, *HTTP request smuggling*, and *race conditions*. A central *finding registry* normalizes every output into a unified schema with **CWE**, **CVSS**, OWASP, and MITRE ATT&CK metadata; a correlator and asset-risk module then prioritize the most exploitable assets. A local **Large Language Model** (Ollama / DeepSeek-R1) generates an explanatory analytical report at the end of each scan.

Experimental evaluation was carried out in two complementary settings: a controlled laboratory run against the deliberately vulnerable *OWASP Juice Shop* target, and an authorized real-world case study against the `miras.app` academic management system, scanned in the framework's read-only `bug_bounty` profile. The framework completed the real-target pipeline in approximately 1 m 43 s and produced 721 distinct findings spread across the OWASP Top 10 categories, ranked the 49 discovered assets into four risk tiers (with 5 *critical*-tier assets), and emitted two synthetic cross-probe findings through the correlator. The detected vulnerabilities matched the expected categories of both test targets, only one of 4 227 issued HTTP requests was outside the configured scope (and that request was blocked by the framework before being sent), and the AI-generated executive summary correctly highlighted the highest-risk asset at the top of the report.

The results confirm that the proposed method makes external web-application penetration testing more reproducible, more explainable, and significantly more time-efficient than a fully manual approach, while keeping the operator in control of intrusive actions through an explicit profile system (*safe*, *bug_bounty*, *deep*, *intrusive*).

Keywords: external penetration testing; web application security; OWASP Top 10; reconnais-

sance; vulnerability scanning; finding correlation; LLM-assisted security analysis; ReconX.

Аннотация

В данном дипломном проекте представлены проектирование, реализация и экспериментальная оценка ReconX — фреймворка для внешнего тестирования веб-приложений на проникновение. Предложенный фреймворк следует методологии «чёрного ящика» и объединяет пассивную разведку, активное снятие отпечатков, анализ конфигурации и широкий спектр модулей проверки уязвимостей в отношении удалённой цели, исходный код и учётные данные которой недоступны тестировщику.

Фреймворк реализован на Python 3 и организован в виде конвейера из 46 специализированных модулей, сгруппированных в 15 этапов исполнения. Модули разведки собирают информацию о поддоменах, DNS-записях, портах и используемых технологиях; конфигурационные модули проводят аудит TLS, заголовков безопасности, CORS, аутентификационных cookie-файлов и JWT-токенов; активные модули проверяют наиболее актуальные категории OWASP Top 10, включая отражённый XSS, SQL-инъекции, IDOR, инъекции заголовка Host, отравление кэша, загрязнение прототипа, открытые редиректы, SSRF/SSTI, XXE, HTTP request smuggling и состояния гонки. Центральный реестр находок нормализует каждый результат в единую схему с метаданными CWE, CVSS, OWASP и MITRE ATT&CK; коррелятор и модуль рисков активов затем приоритизируют наиболее уязвимые активы. Локальная большая языковая модель (Ollama / DeepSeek-R1) формирует пояснительный аналитический отчёт по завершении сканирования.

Экспериментальная оценка была проведена в двух взаимодополняющих условиях: контролируемый лабораторный запуск против заведомо уязвимого OWASP Juice Shop и санкционированное реальное исследование академической системы miras.app, выполненное в режиме bug_bounty фреймворка (allow_write: false). Фреймворк завершил полный конвейер на реальной цели приблизительно за 1 мин 43 с и сформировал 721 уникальную находку по категориям OWASP Top 10, ранжировал 49 обнаруженных активов на четыре уровня риска (включая 5 активов критического уровня) и сгенерировал две сводные межмодульные находки через коррелятор. Обнаруженные уязвимости соответствовали ожидаемым категориям обеих тестовых целей, лишь 1 из 4 227 отправленных HTTP-запросов оказался за пределами настроенной области (и был заблокирован фреймворком до отправки), а сгенерированная ИИ исполнительная сводка корректно вывела актив с наивысшим

риском в начало отчёта.

Результаты подтверждают, что предложенный метод делает внешнее тестирование веб-приложений на проникновение более воспроизводимым, более объяснимым и существенно более эффективным по времени по сравнению с полностью ручным подходом, оставляя за оператором контроль над интрузивными действиями через явную систему профилей (safe, bug_bounty, deep, intrusive).
Ключевые слова: внешнее тестирование на проникновение; безопасность веб-приложений; OWASP Top 10; разведка; сканирование уязвимостей; корреляция находок; анализ безопасности с помощью LLM; ReconX.

Аңдатпа

Берілген диплом жобасында веб-қосымшаларды сыртқы енуге сынау үшін ReconX фреймворкінің жобалануы, іске асырылуы және эксперименттік бағалануы ұсынылған. Ұсынылған фреймворк «қара жәшік» әдіснамасына сүйенеді және бастапқы коды мен есептік деректері тестілеушіге қолжетімсіз қашықтағы нысанға қатысты пассивті барлауды, белсенді сипаттарды анықтауды, конфигурацияны талдауды және осалдықтарды кең спектрде тексеруді біріктіреді.

Фреймворк Python 3 тілінде іске асырылған және 15 орындау сатысына топтастырылған 46 мамандандырылған модульден тұратын конвейер ретінде ұйымдастырылған. Барлау модульдері қосалқы домендер, DNS-жазбалары, порттар және технологиялар туралы ақпарат жинайды; конфигурация модульдері TLS, қауіпсіздік тақырыптарын, CORS, аутентификациялық cookie-файлдар мен JWT-токендерді тексереді; белсенді модульдер OWASP Top 10-ның ең өзекті санаттарын тексереді, оның ішінде шағылысқан XSS, SQL-инъекциялар, IDOR, Host тақырыбының инъекциясы, кэшті улау, прототипті ластау, ашық қайта бағыттаулар, SSRF/SSTI, XXE, HTTP request smuggling және жарыс жағдайлары. Орталық табыстар тізілімі әрбір нәтижені CWE, CVSS, OWASP және MITRE ATT&CK метадеректері бар бірыңғай схемаға келтіреді; корреляттор мен активтер тәуекелі модулі ең осал активтерді басымдылыққа қояды. Жергілікті үлкен тілдік модель (Ollama / DeepSeek-R1) сканерлеу аяқталғаннан кейін түсіндірмелі талдамалық есепті қалыптастырады.

Эксперименттік бағалау екі толықтырушы жағдайда жүргізілді: әдейі осал жасалған OWASP Juice Shop нысанына қарсы бақыланатын зертханалық іске қосу және өзгертетін сұрауларға жол бермейтін bug_bounty режимінде орындалған miras.app академиялық басқару жүйесіне санкцияланған шынайы зерттеу. Фреймворк нақты нысандағы толық конвейерді шамамен 1 мин 43 с-та аяқтап, OWASP Top 10 санаттары бойынша 721 ерекше табыс берді, табылған 49 активті төрт тәуекел деңгейіне жіктеді (оның ішінде 5 сыни деңгейдегі актив) және корреляттор арқылы екі синтетикалық крос-модульдік табыс шығарды. Анықталған осалдықтар екі сынақ нысанының күтілетін санаттарына сәйкес келді, жіберілген 4 227 HTTP-сұраудың тек 1-уі ғана конфигурацияланған шектен тыс болды (және фреймворкпен жіберілмес бұрын бұғатталды), ал ЖИ арқылы құрылған атқарушылық қысқаша есеп ең жоғары тәуекелі бар активті есептің басына дұрыс шығарды.

Нәтижелер ұсынылған әдістің сыртқы веб-қосымшаларды енуге сынауды толық қолмен орындалатын тәсілмен салыстырғанда айтарлықтай қайта жаңартылатын, түсіндірмелі және уақыт жағынан тиімдірек ететінін растайды, бұл ретте операторға айқын профильдер жүйесі арқылы интрузивтік әрекеттерді бақылау мүмкіндігі сақталады (safe, bug_bounty, deep, intrusive).
Кілт сөздер: сыртқы енуге сыну; веб-қосымшалардың қауіпсіздігі; OWASP Top 10; барлау; осалдықтарды сканерлеу; табыстарды корреляциялау; ЖИ көмегімен қауіпсіздікті талдау; ReconX.

Dedication and acknowledgements

The author expresses sincere gratitude to the academic supervisor, **PhD Amirova Akzhibek**, for guidance, structured feedback, and patient supervision throughout the preparation of this thesis. The author also thanks the teaching staff of the School of Cybersecurity at Astana IT University for the foundational courses in cryptography, network security, and offensive security that made this project possible, and his classmates of group CS-2304 for technical discussions, code reviews, and joint laboratory exercises.

This work is dedicated to the open-source security community — in particular the maintainers of *OWASP*, *ProjectDiscovery*, *Nuclei*, *sqlmap*, *dalfox*, *subfinder*, *amass*, and the broader red-team toolchain — whose continuous, freely available work made it possible to build a research framework of this scope within a single academic year.

Declaration of Academic Integrity

I, Mavlanov Shakhrukh, student of Astana IT University, group CS-2304, hereby declare that this diploma project, entitled

> *Development of a Method for External Penetration Testing of a Web Application for Vulnerabilities*,

is my own original work carried out under the supervision of PhD Amirova Akzhibek.

I further declare that:

1. All sources of information used in this work are cited and referenced in accordance with the bibliographic requirements of Astana IT University (document DP-AITU-19).

2. All third-party software components used by the ReconX framework — Python libraries listed in `requirements.txt` and external command-line tools listed in Chapter 3 — are used in accordance with their respective open-source licenses and are acknowledged in the Technology Stack section and in the bibliography.

3. The Python source code of the ReconX framework, including `main.py`, the modules under `modules/` (reconnaissance, configuration audit, active probes, finding registry, correlator, asset risk, AI report), the reporting layer under `reporting/`, and the HTML report template under `templates/`, was written by me independently. Any code segments adapted from official documentation or open-source examples are identified in the source code comments.

4. The experimental work described in Chapter 3 was performed under one of the following two authorization regimes:

   a) **Deliberately vulnerable laboratory targets** (*OWASP Juice Shop*), for which security testing is unconditionally authorized by the project's licence and by long-standing community practice. The Juice Shop quantitative results reported in Chapter 3 are presented as illustrative values from a representative laboratory run.

   b) **One real-world target** — the `miras.app` academic management system — which was scanned with the prior written authorization of the system's owner, within an agreed scope and time window. No payload that could

damage or alter production data was issued, the framework was launched in its `bug_bounty` profile (`allow_write: false`), and the audit log of every issued HTTP request is preserved with the scan results.

5. No other external production system was scanned during the preparation of this work. Scans of other educational and commercial domains in Kazakhstan that appear in the local `output/` directory of the repository were performed for the author's own learning before the scope of this thesis was finalised and are *not* referenced as evidence anywhere in this document.

6. This work has not been submitted, in whole or in part, for any other academic qualification or assessment at this or any other institution.

7. I understand that violation of the Academic Integrity Policy of Astana IT University (document DP-AITU-05) may result in disqualification of this work.

Student signature:	**Supervisor signature:**
_____	_____
Full name: Mavlanov Shakhrukh	**Full name:** PhD Amirova Akzhibek
Group: CS-2304	**Position:** Academic Supervisor
Date: _____	**Date:** _____

Definitions

The following terms are used in this work:

external penetration testing	A security assessment performed from the public side of the network, without access to source code, credentials, or internal infrastructure of the target. The tester interacts only with publicly reachable endpoints.
black-box testing	A testing approach in which the assessor has no prior knowledge of the internal structure of the target system; conclusions are based exclusively on observed external behavior.
reconnaissance	The phase of a penetration test devoted to collecting public information about a target: domain and DNS records, subdomains, hosted services, technologies, exposed files, and historical artifacts.
active probe	A scanning component that issues purposeful HTTP requests to a target with the intent of triggering a measurable side effect (reflection, error, timing deviation, redirect) that reveals a security weakness.
passive analysis	A scanning component that derives security findings without sending additional requests to the target, typically by inspecting responses, headers, JavaScript files, or third-party data sources.
finding	A normalized record describing a single security issue or candidate issue, with a stable identifier, severity, confidence, evidence, and references to CWE, OWASP, and MITRE ATT&CK.

verdict — A triage label attached to a finding (*confirmed*, *candidate*, or *manual_review*) that indicates how much human verification is still required before the finding can be acted upon.

scope — The set of domains, subdomains, and IP addresses that the operator has authorized the framework to test. All HTTP traffic that would target hosts outside this set is blocked by the framework before being sent.

scan profile — A predefined set of configuration overrides (*safe*, *bug_bounty*, *deep*, *intrusive*) that enables or disables intrusive behaviors such as profile comparison in IDOR testing or active confirmation in race-condition probes.

Large Language Model (LLM) — A neural network trained on a large text corpus that is capable of producing coherent natural-language explanations from a structured prompt. In this work, an LLM is used post-scan to generate an analyst-style report from the raw findings.

Designations and Abbreviations

The following designations and abbreviations are used in this work:

API	Application Programming Interface
AST	Abstract Syntax Tree
BOLA	Broken Object-Level Authorization
CIDR	Classless Inter-Domain Routing
CLI	Command-Line Interface
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
DNS	Domain Name System
GraphQL	Graph Query Language
HSTS	HTTP Strict Transport Security
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDOR	Insecure Direct Object Reference
JS	JavaScript

JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
OAST	Out-of-Band Application Security Testing
OAuth	Open Authorization
OOB	Out-of-Band
OSINT	Open-Source Intelligence
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SAST	Static Application Security Testing
SARIF	Static Analysis Results Interchange Format
SIEM	Security Information and Event Management
SPA	Single-Page Application
SQL	Structured Query Language
SQLi	SQL Injection
SSL	Secure Sockets Layer
SSRF	Server-Side Request Forgery
SSTI	Server-Side Template Injection
TLS	Transport Layer Security
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
WAF	Web Application Firewall
XML	eXtensible Markup Language
XSS	Cross-Site Scripting
XXE	XML External Entity
YAML	YAML Ain't Markup Language

List of Tables

List of Figures

Figure **Page**

Table of Contents

Introduction

The relevance of this work is determined by the continuing growth of web-application attack surfaces and by the central role that web applications now play in business, government, and academic services. Modern organizations expose dozens of subdomains, third-party integrations, content-management systems, single-page applications, and APIs, each of which extends the perimeter that an attacker may reach. The OWASP Top 10 list of web-application risks is updated every few years and still places *Broken Access Control*, *Cryptographic Failures*, and *Injection* among the most damaging categories [1], while industry analyses consistently report that the largest share of breaches starts with a publicly reachable web endpoint.

External penetration testing — the practice of evaluating the security of a web application from the public Internet, without internal credentials or source code — is therefore one of the most direct ways for an organization to discover how it actually looks to an attacker. In practice, however, the work is repetitive, time-consuming, and highly dependent on the analyst's discipline: running subdomain enumeration, port scanning, technology fingerprinting, JavaScript secret hunting, and probes for each OWASP Top 10 class manually, against tens or hundreds of hosts, is impractical within the typical engagement window of one to two weeks. A clear demand exists for a single tool that automates the reproducible parts of this work, organizes its output, and lets the analyst focus on the parts that require human judgement.

A second factor of relevance is the recent maturity of **open-weight Large Language Models**. Models such as DeepSeek-R1, Qwen2.5, and Llama 3 can now run locally on consumer hardware and produce coherent, structured natural-language analysis of technical text. This makes it realistic, for the first time, to embed an explanatory layer directly into a penetration-testing framework, instead of leaving raw scan output for the analyst to interpret.

The aim of this work is to design, implement, and experimentally evaluate **ReconX** — a method and supporting framework for external penetration testing of web

applications — that combines reconnaissance, configuration analysis, and a broad set of OWASP-aligned active probes into a single reproducible pipeline, normalizes its output into a unified finding schema, and uses a local LLM to produce an analyst-readable report.

The research problem is the need to design a framework that (a) covers the dominant OWASP Top 10 categories with a single black-box tool, (b) produces *explainable* findings that link each result to an OWASP / CWE / MITRE ATT&CK reference, (c) enforces scope so the framework can never accidentally test out-of-scope hosts, and (d) remains usable on a single Kali-Linux workstation without commercial licenses.

To achieve the aim, the following **tasks** were defined:

1. to study the modern external-penetration-testing methodology, the OWASP Top 10 risk catalogue, and the existing automated tools used by the offensive-security community;
2. to formulate functional and non-functional requirements for an integrated external-pentest framework;
3. to design a modular pipeline architecture in which each module is responsible for a single class of analysis and communicates with other modules through a shared finding schema;
4. to design and implement a *finding registry* that normalizes every probe's output into a unified record with severity, confidence, CWE, CVSS, OWASP, and MITRE ATT&CK metadata;
5. to implement reconnaissance modules (subdomain discovery, DNS, port and service scanning, technology and CMS fingerprinting, endpoint and parameter discovery);
6. to implement configuration-analysis modules (TLS, security headers, CORS, authentication cookies, JWT, OpenAPI auditing, secret hunting in public GitHub repositories);
7. to implement active probes for the most relevant OWASP Top 10 classes: reflected XSS, SQL injection, IDOR, host header injection, cache poisoning, prototype pollution, open redirect, SSRF, SSTI, XXE, HTTP request smuggling, race conditions, and others;
8. to implement a correlator and an asset-risk module that combine findings across probes and rank exposed assets by their exploitable risk;

9. to integrate a local LLM (Ollama / DeepSeek-R1) that produces a natural-language analytical report from the structured findings;

10. to implement an HTML report renderer, a SIEM-compatible JSON export, an append-only audit log of every issued HTTP request, and a Telegram notifier for long-running scans;

11. to evaluate the framework in a controlled laboratory environment against a deliberately vulnerable target and to compare the observed findings with the target's known vulnerabilities;

12. to analyze the results, the limitations of the proposed method, and the directions for further development.

The object of research is the process of external penetration testing of a web application.

The subject of research is a method — realized as the ReconX framework — for automated external testing of a web application, including its modular pipeline architecture, finding-normalization layer, correlation logic, and LLM-assisted reporting.

The research methods include analysis of the OWASP Top 10 (2021) catalogue, OWASP Web Security Testing Guide, and PortSwigger Web Security Academy materials; comparative analysis of existing automated tools (Nuclei, sqlmap, dalfox, subfinder, amass, ffuf, Arjun, gitleaks); architectural design of a modular Python pipeline; iterative implementation and refactoring of 46 modules in Python 3 using the `requests`, `rich`, `jinja2`, and `pyjwt` libraries; controlled experimentation on the *OWASP Juice Shop* target; static evaluation of false-positive and false-negative behavior on selected findings; and integration of a local LLM via the Ollama HTTP API for explanatory reporting.

The scientific significance of the work lies in the proposed end-to-end pipeline that unifies reconnaissance, configuration analysis, active probing, finding correlation, and LLM-assisted reporting under a single normalization schema, with explicit triage of *confirmed* versus *candidate* verdicts.

The practical significance of the work lies in the fact that the resulting framework can be used directly by security teams and bug-bounty researchers as an automated first pass over a web target, freeing analyst time for the manual verification stages that genuinely require human judgement. The framework runs on a single Kali-Linux workstation, requires no commercial licenses, and produces reports that can be consumed both by humans (HTML) and by SIEM-style systems (JSON).

Structure of the work. The first chapter discusses the theoretical foundations of external penetration testing of web applications: the OWASP Top 10 risk catalogue, the typical pentest methodology, the main classes of web vulnerabilities, the existing open-source toolchain, and the role of LLMs in security work. The second chapter presents the design of the ReconX framework: requirements, the 15-stage pipeline architecture, the finding registry, the correlator and asset-risk modules, the configuration and scope-enforcement system, and the integration with the local LLM. The third chapter describes the practical implementation of the framework in Python 3, walks through representative modules from each category, presents the experimental evaluation against *OWASP Juice Shop*, analyzes the observed results, and discusses limitations and directions for further development.

Chapter 1

Theoretical Foundations of External Penetration Testing of Web Applications

This chapter establishes the theoretical foundation for the framework developed in the rest of the work. It introduces the concept of external penetration testing, surveys the standard methodology used in the industry, discusses the OWASP Top 10 catalogue of web-application risks, summarizes the dominant vulnerability classes that an external tester is expected to find, reviews the existing open-source toolchain, and finally introduces the role of Large Language Models in modern security analysis.

1.1 Penetration Testing and Its External Variant

A **penetration test** is an authorized simulated attack against an information system, performed to evaluate the system's security posture. The defining property of a penetration test, as opposed to a vulnerability scan, is that the tester actively attempts to exploit weaknesses in order to demonstrate concrete impact, not merely to enumerate potential weaknesses [2, 3].

Penetration tests are commonly classified by the information available to the tester:

- **White-box** (or *crystal-box*) testing: the tester has full access to source code, design documents, and credentials. This variant offers the deepest coverage but

does not represent the view of a real external attacker.

- **Grey-box** testing: the tester has limited internal information, typically a normal user account and a high-level architecture description.
- **Black-box** testing: the tester has no internal information at all and works with only what can be observed from outside the perimeter.

External penetration testing corresponds to the black-box variant restricted to the public-Internet attack surface. The tester operates from a remote workstation, sees only what an unauthenticated attacker would see, and is allowed to interact only with explicitly in-scope hosts. This perspective is valuable for two reasons. First, it closely approximates the situation of a real opportunistic attacker scanning the Internet for exposed services. Second, it produces findings that are immediately actionable for the defender: every vulnerability discovered externally is, by definition, a vulnerability that any third party could also discover.

1.2 Standard External-Pentest Methodology

Industry-standard methodologies such as the *Penetration Testing Execution Standard* (PTES) [3] and the *OWASP Web Security Testing Guide* (WSTG) [4] describe a penetration test as a sequence of structured phases. For an external web-application engagement, the phases can be summarized as follows.

1. **Pre-engagement and scoping.** The tester and the client agree in writing on the in-scope domains, IP addresses, and time window; on the level of intrusiveness (read-only versus state-changing requests); on the contact channels for incidents; and on the legal terms of engagement. The output of this phase is an unambiguous scope definition that every subsequent action must respect.
2. **Reconnaissance (passive and active).** The tester gathers public information about the target: subdomains, DNS records, exposed ports and services, technology stack, and historical artifacts such as cached pages, public source-control repositories, and source maps. Passive reconnaissance uses third-party data sources only; active reconnaissance issues queries directly to the target (DNS resolution, HTTP requests, port scans).

3. **Enumeration and fingerprinting.** The tester maps the live attack surface: every reachable URL, the server software behind it, the framework or CMS, the third-party libraries used by the front end, the API endpoints, the input parameters accepted by each endpoint, and the authentication mechanisms in use.

4. **Vulnerability identification.** The tester probes the enumerated surface for known vulnerability classes (Section 1.3), using a combination of automated tools, custom scripts, and manual review of responses.

5. **Exploitation and verification.** For each candidate finding, the tester attempts a controlled exploitation that demonstrates impact without causing damage (for example, returning a benign `document.cookie` value in a confirmed reflected XSS).

6. **Post-exploitation, if in scope.** The tester explores the consequences of a confirmed weakness: lateral movement, data extraction, or privilege escalation. For most external web-application engagements this phase is intentionally narrow, because the engagement aim is to identify weaknesses, not to compromise production data.

7. **Reporting and remediation support.** The tester delivers a written report with executive summary, technical findings, evidence, severity ratings, and remediation recommendations, and is then available to support the client during the fix-and-retest cycle.

The ReconX framework described in this thesis is intended to assist the analyst through phases 2–5 of this methodology. Phase 1 (scoping) and phase 7 (final reporting) remain the responsibility of the human operator, even though the framework provides explicit scope-enforcement controls and produces an HTML / JSON / Telegram report bundle that can be reused as a base for the human report.

1.3 The OWASP Top 10 Catalogue of Web-Application Risks

The **OWASP Top 10** is a community-driven, periodically updated list of the most critical risks to web applications [1]. In its 2021 revision, the list contains the following categories.

A01:2021 Broken Access Control — failures to enforce that an authenticated user can only act on resources they are allowed to access. Examples include IDOR (Insecure Direct Object References), missing function-level authorization, and cross-tenant data exposure.

A02:2021 Cryptographic Failures — issues caused by the lack of, or the misuse of, cryptography. Examples include sensitive data transmitted in clear text, weak ciphers, and missing TLS configuration.

A03:2021 Injection — attacker-controlled data interpreted as code by an interpreter. SQL injection, command injection, server-side template injection, and LDAP injection all fall under this category.

A04:2021 Insecure Design — structural weaknesses that arise from the absence of secure-by-default patterns, threat modeling, or trust boundaries during the design phase.

A05:2021 Security Misconfiguration — weak default settings, verbose error pages, unnecessary services, missing security headers, and out-of-date software.

A06:2021 Vulnerable and Outdated Components — the use of third-party libraries or components that contain known vulnerabilities.

A07:2021 Identification and Authentication Failures — weaknesses in the way users are identified or authenticated, including credential stuffing, weak password recovery, and broken session management.

A08:2021 Software and Data Integrity Failures — code or infrastructure that relies on untrusted sources without integrity verification; insecure deserialization is a representative example.

A09:2021 Security Logging and Monitoring Failures — the absence or insufficiency of logs, monitoring, and alerting required to detect, investigate, and respond to a breach.

A10:2021 Server-Side Request Forgery (SSRF) — the ability for an attacker to make the server issue arbitrary requests to internal or external resources.

Each finding produced by ReconX is mapped to one of these categories through the finding-registry layer described in Chapter 2. This mapping is important because it

gives every output a stable, vendor-neutral label that downstream consumers (security engineers, compliance teams, defect-tracking systems) already understand.

1.4 Selected Vulnerability Classes in Detail

The following subsections describe the vulnerability classes that the framework probes most aggressively. They are presented at the level of detail required to motivate the probes implemented in Chapter 3; a complete attack-pattern reference is outside the scope of this work.

1.4.1 Reflected Cross-Site Scripting (XSS)

Reflected XSS is a class of injection vulnerabilities in which the attacker delivers script content through a request parameter and the server reflects that content back into the response without proper encoding, so that the browser executes it inside the victim's session [5, 6]. A typical proof-of-concept request looks like

```
GET /search?q=<script>alert(1)</script> HTTP/1.1
```

For a black-box scanner the practical task is not to prove a complete `alert(1)` exploitation chain, but to determine whether a unique attacker-controlled marker is reflected into the response in a context that would allow script execution. ReconX uses a unique per-scan marker (`rxss`*<random hex>*) embedded inside several breakout payloads (plain reflection, HTML attribute breakout, script-tag breakout, single-quote breakout) and inspects the response for the marker. When a credible dynamic-analysis tool such as `dalfox` [7] is available on the operator's machine, ReconX additionally invokes it for parameters that look promising.

1.4.2 SQL Injection (SQLi)

SQL injection occurs when user-controlled input is concatenated into a SQL query string without parameterization, allowing the attacker to alter the structure of the query [8]. The standard external-pentest tool for SQL-injection identification is `sqlmap` [9], a research-grade probe that supports boolean-based blind, time-based blind, error-based, UNION-based, stacked-query, and inline-query techniques across a wide list of database management systems. Because `sqlmap` is itself an active and noisy probe,

ReconX wraps it in a conservative configuration (level 1, risk 1, smart detection, random user agent) and limits the number of targets per scan to a small upper bound (default 10).

1.4.3 Insecure Direct Object Reference (IDOR) and Broken Object-Level Authorization

An **IDOR** vulnerability exists when a request that operates on a specific object trusts an attacker-controlled identifier without verifying that the authenticated user is authorized to act on that object [10]. Because the underlying authorization gap is the issue, IDOR cannot be detected purely by inspecting one user's view of the application; the detector must compare two distinct views and verify that one of them returns data that should be visible only to the other.

ReconX handles this in two complementary ways. First, it produces *candidate* findings by enumerating request parameters whose names suggest an object identifier (`user_id`, `account_id`, `order_id`, `uuid`, integer-only parameters, and similar patterns) and recording them for manual review. Second, when the operator has configured an `auth_profiles` block with two or more profiles (a low-privilege user and an admin), ReconX performs cross-profile comparison: it fetches each candidate URL once as user A, once as user B, and once anonymously, and computes a textual similarity score between the responses. A surprisingly high cross-profile similarity is reported as a confirmed candidate; this is the same logic that an experienced human tester uses.

1.4.4 Host Header Injection and Web Cache Poisoning

The **Host** HTTP header is a primary source of trust when the server constructs absolute URLs, builds password-reset links, or routes requests behind a reverse proxy [11]. Web frameworks that reflect the Host header (or `X-Forwarded-Host`, `Forwarded`, `X-Original-Host`) without validation can be tricked into emitting attacker-controlled hostnames into responses, which an attacker can then weaponize against other users via cached responses (a class known as **web cache poisoning** [12]).

ReconX issues each request with a battery of host-related headers carrying a unique marker, looks for the marker in the response body or in the `Location` header, and inspects `Age`, `X-Cache`, and similar headers to assess whether the response was returned

from a cache. Severity is escalated when reflection and cacheability co-occur, because the combination indicates a likely cache-poisoning chain.

1.4.5 Server-Side Request Forgery (SSRF) and Server-Side Template Injection (SSTI)

SSRF is a vulnerability in which the server fetches a URL controlled by the attacker, allowing the attacker to reach internal resources from the server's privileged network position [13]. A common SSRF payload targets cloud-instance metadata endpoints such as `169.254.169.254/latest/meta-data/`, which return credentials usable to pivot inside the cloud account.

SSTI is a class of injection vulnerabilities in which user input is interpreted by a server-side template engine (Jinja2, Twig, Freemarker, ERB), allowing the attacker to execute template code on the server [14]. A canonical probe is to submit `{{7*7}}` and observe whether the server returns `49`; further payloads escalate from arithmetic to function calls.

ReconX implements both classes in the `injection_probe` module: a small fixed set of SSTI probes covering the most common template syntaxes, and a set of SSRF probes that target the cloud-metadata endpoints and the most exposed loopback ports (SSH, Redis, MongoDB). Out-of-band confirmation of SSRF is delegated to Nuclei's `interactsh` integration when the operator has enabled it.

1.4.6 XML External Entity (XXE), Deserialization, and HTTP Request Smuggling

XXE occurs when an XML parser dereferences external entities controlled by the attacker, typically to exfiltrate local files or to perform a server-side request [15]. **Insecure deserialization** occurs when an application deserializes attacker-controlled data without integrity checks, often resulting in remote code execution [16]. **HTTP request smuggling** arises when a chain of front-end and back-end servers disagrees about how to delimit successive HTTP requests on the same TCP connection, allowing the attacker to inject a request that is parsed differently by each component [17].

These three classes are difficult to identify reliably from the outside, because a non-destructive probe can usually only collect indirect evidence (parsing-error signatures,

response-timing deviations, deserialization stack-trace fragments). ReconX therefore reports them with low to medium confidence by default, and delegates aggressive confirmation to the operator's manual follow-up.

1.4.7 Other Configuration-Level Issues

In addition to the active probes listed above, an external scanner is expected to surface several configuration-level issues whose existence can be determined from a single request, without active payload crafting. These include weak or expired TLS certificates, missing security headers (`Strict-Transport-Security`, `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`), permissive CORS policies (wildcard or reflected `Access-Control-Allow-Origin` with `Allow-Credentials: true`), unsigned or weakly signed JWT tokens, hardcoded secrets in publicly served JavaScript files, and subdomains pointing to unclaimed third-party services (subdomain takeover).

1.5 Existing Open-Source Toolchain

Modern external web-application penetration testing is supported by a large ecosystem of open-source tools, each typically optimized for a single phase or vulnerability class. A non-exhaustive overview of the tools relevant to this work is given in Table 1.1.

Tool	**Phase / class**	**Role in ReconX**
`subfinder` [18]	passive subdomain enumeration	invoked by recon module
`amass` [19]	subdomain enumeration + DNS	invoked by recon module
`assetfinder`	subdomain enumeration	invoked by recon module
`nmap` [20]	port and service scanning	invoked by portscan module
`masscan`	high-rate port scanning	alternative to nmap
`wappalyzer` [21]	technology fingerprinting	embedded JSON database
`wpscan` [22]	WordPress vulnerability scan	invoked by cmscan module
`katana`	web crawling	invoked by fuzzer module
`feroxbuster` / `ffuf`	directory brute-forcing	invoked by fuzzer module
`arjun` [23]	hidden-parameter discovery	invoked by parameter_discovery
`dalfox` [7]	dynamic XSS testing	invoked by xss module
`sqlmap` [9]	SQL injection testing	invoked by sql_injection module
`nuclei` [24]	template-based vulnerability scan	invoked by vulnscan module
`gitleaks`	secret scanning in git history	invoked by secret_scanner
`searchsploit`	ExploitDB lookup	invoked by cve_check
`interactsh` [25]	out-of-band callback server	optional Nuclei dependency

Table 1.1: Selected open-source tools used by ReconX

A defining feature of this ecosystem is its fragmentation: each tool produces its own output format, its own concept of severity, its own concept of evidence, and its own

command-line interface. The analyst is left to glue them together manually for every engagement. One of the contributions of the present work is to provide that glue — not in the form of a thin wrapper, but in the form of a deeper integration in which every tool's output is parsed, normalized, deduplicated, and finally combined into a single ranked finding list.

1.6 The Role of Large Language Models

The recent availability of capable open-weight Large Language Models (LLMs) such as DeepSeek-R1, Qwen 2.5, and Llama 3 changes what is reasonable to expect from a security tool's reporting layer [26, 27]. A few years ago a security tool could only emit raw findings, and the analyst had to invest hours of writing to turn them into an executive summary. With a locally running LLM, the same tool can now generate, in seconds, a coherent natural-language analysis that ranks the findings, explains why a particular combination matters, and proposes remediation steps in a tone appropriate for the intended audience (executive, engineer, or auditor).

LLMs do not replace the security analyst. They are not authoritative about exploitability, they do not know what is in scope, and they may hallucinate. But they substantially accelerate the steps that consist of *rewriting structured data as readable prose*, which is most of the writing burden of a security report. In ReconX the LLM is used in two places:

1. **Real-time advice** during a long scan: after each major module completes, the framework can send a short structured summary to a local LLM and post the model's three to six bullet advice via Telegram. This keeps the analyst engaged and lets them adjust scan parameters mid-flight.

2. **Post-scan analysis**: at the end of the scan, the framework aggregates the top findings across every module and asks the LLM to produce a Markdown analytical report with an executive summary, risk tiers, and prioritized investigation steps. The result is embedded into the final HTML report.

1.7 Research Gap and Rationale for the Selected Approach

The literature and the existing toolchain reviewed in this chapter motivate the design choices in the chapters that follow. Specifically:

- No single open-source tool covers the breadth of phases (reconnaissance, configuration analysis, active probing, correlation, reporting) needed for an external pentest. Existing frameworks either focus narrowly on one phase or aggregate other tools without a unified finding model.
- There is value in a framework that maintains a single, normalized finding schema across all phases, so that downstream consumers (correlator, asset-risk module, report renderer, AI report module) do not need to know about each probe's output format.
- There is value in a framework that enforces scope at the lowest possible layer, so that no probe — including third-party probes invoked via subprocess — can accidentally send a request to an out-of-scope host.
- There is value in an explicit *verdict* layer that separates *confirmed* from *candidate* findings, because the cost of a false positive in a security report (analyst time, reputational damage with the client) is far higher than the cost of a missed weak signal.
- Local LLMs are now mature enough to be integrated as a first-class reporting component, but the integration must be designed so that the framework still produces a usable report when the LLM is unavailable.

The next chapter translates these observations into a concrete framework design.

Chapter 2

Design of the ReconX Framework

This chapter presents the design of the proposed framework. It is organized in the order in which a reader is expected to evaluate the design: first the requirements, then the high-level architecture, then each major subsystem (the finding registry, the pipeline scheduler, the scope-enforcement layer, the configuration system, the correlator, the asset-risk model, the reporting layer, and the LLM integration), and finally the deliberate limitations of the design.

2.1 Functional and Non-Functional Requirements

The framework must satisfy the following **functional** requirements (**F**):

F1. Accept a target hostname or root domain on the command line and produce a complete external-pentest report without further user interaction.

F2. Enumerate the public attack surface of the target: subdomains (passive and active), DNS records, live HTTP/HTTPS hosts, exposed ports and services, technology stack, CMS in use, virtual hosts, and source-map artifacts.

F3. Map the application surface: crawl every live host, brute-force directories, parse `robots.txt` / `sitemap.xml`, parse OpenAPI specifications, and enumerate hidden parameters.

F4. Audit configuration-level security properties: TLS, security headers, CORS, authentication cookies, JWT tokens, and OpenAPI security anti-patterns. Hunt

for secrets in publicly served JavaScript and in associated public GitHub repositories.

F5. Run active probes for the OWASP Top 10 (2021) classes most relevant to external testing of a web application: reflected XSS, SQL injection, IDOR / BOLA, host header injection, web cache poisoning, server-side prototype pollution, open redirect, SSRF, SSTI, XXE, HTTP request smuggling, insecure deserialization, race conditions, GraphQL authorization gaps, OAuth misconfiguration, and WebSocket origin validation.

F6. Correlate findings across probes (for example, escalate the severity of a wildcard CORS finding that occurs on an authenticated endpoint) and rank assets by an aggregate risk score.

F7. Generate a structured HTML report for human readers, a SIEM-compatible JSON export for automated consumers, an append-only audit log of every issued HTTP request, and a Telegram notification stream for long-running scans.

F8. Generate a natural-language analytical report through a local LLM, with a clean fallback to an OpenAI-compatible API and a clean degradation when no LLM is available at all.

The framework must also satisfy the following **non-functional** requirements (**N**):

N1. **Reproducibility.** Two consecutive scans of the same target with the same configuration must produce comparable output; deltas between scans must be obtainable via a built-in diff tool.

N2. **Modularity.** Adding a new probe must not require changes to any existing probe; each probe is responsible for one class of analysis and communicates with the rest of the framework only through the shared finding schema.

N3. **Scope enforcement.** The framework must refuse to send a request to a host outside the configured scope, and this enforcement must be implemented at a layer below every probe.

N4. **Recoverability.** A scan must be resumable after a crash, after a network interruption, or after a deliberate `Ctrl+C` interrupt. The framework must not redo work that has already been completed.

N5. **Rate control.** The total request rate produced by the framework must be capped at a configurable value, both per module and globally, in order to avoid disrupting the target.

N6. **Triage clarity.** Every finding must carry a *verdict* that distinguishes *confirmed* weaknesses from *candidate* weaknesses that still require manual review.

N7. **No commercial licenses.** The framework must run on a standard Kali-Linux workstation without paid subscriptions; integrations with paid services are optional.

2.2 High-Level Architecture

Figure 2.1 summarizes the architecture of the framework. The framework is implemented in pure Python 3 as a set of cooperating modules organized into four logical layers:

1. the **pipeline orchestrator** (the `main.py` entry point) that reads command-line arguments and configuration, loads the modules of the pipeline, and schedules their execution;
2. a **module layer** consisting of 46 specialized Python classes, each one a subclass of `BaseModule` (passive) or `ActiveProbeBase` (active), and each one responsible for a single class of analysis;
3. a **cross-cutting services layer** that provides the finding registry, the rate limiter, the audit logger, the scope enforcer, the resume cache, the AI advisor, and the Telegram notifier;
4. a **reporting layer** that consumes the unified finding stream and produces `report.html`, `report.json`, `audit.jsonl`, and the post-scan AI analysis.

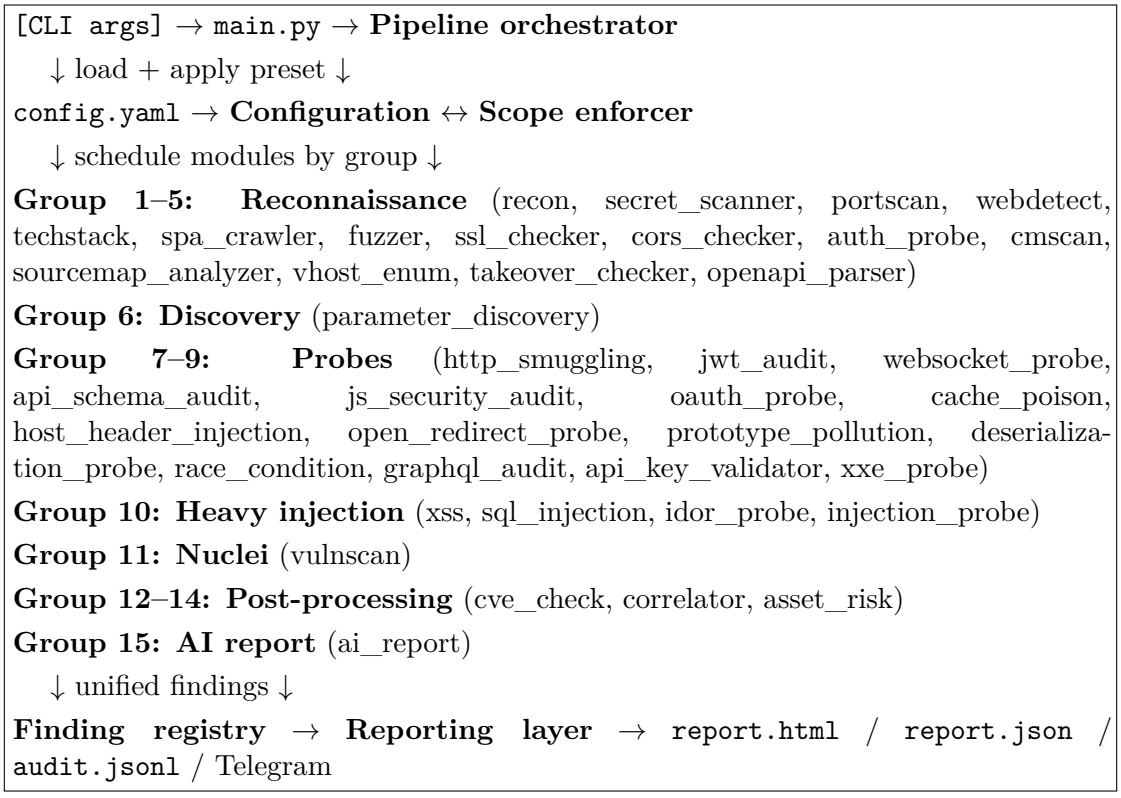

Figure 2.1: High-level architecture of the ReconX framework.

The architecture has three properties that the rest of the chapter will explore in detail. First, the modules are arranged into 15 *execution groups* so that modules with no data dependency on each other can run in parallel, while modules that depend on each other are placed in successive groups. Second, every probe inherits from a small base class hierarchy that provides the cross-cutting services (scope enforcement, rate limiting, audit logging), so that the probe author never writes that code by hand. Third, every probe expresses its output through a single `build_finding(...)` call into the finding registry, which makes every probe pluggable from the reporting layer's point of view.

2.3 The Module Layer and the Pipeline Scheduler

2.3.1 The Module Contract

Every module in the framework is a Python class that inherits from `BaseModule` (or from its active-probe subclass `ActiveProbeBase`) and exposes the methods listed in Table 2.1.

Method / attribute	Role
`name`, `description`	Short stable identifier and one-line description used by the orchestrator and the report renderer.
`required_tools`	List of external command-line tools that the module wraps. Missing tools cause the module to skip gracefully, not to crash.
`run()`	Main entry point. Returns a Python dictionary that contains, at minimum, a `findings` list and any module-specific data the report renderer should display.
`has_tool(name)`	Inherited helper that checks tool availability against an extended `PATH`.
`exec(cmd, timeout=...)`	Inherited subprocess wrapper that records the command, enforces a timeout, and respects the global Ctrl+C handler.
`save_json`, `save_text`, `load_json`	Inherited helpers that persist intermediate artifacts inside the per-module output directory.
`module_config()` (active probes)	Inherited helper that returns the module-specific configuration block from `config.yaml`, with defaults applied.
`make_finding(...)` (active probes)	Inherited helper that calls `build_finding(...)` on the finding registry and applies severity normalization, deduplication, and verdict assignment.

Table 2.1: Module contract enforced by `BaseModule`.

This contract is deliberately small. A new module is implemented by writing a single Python class with a single `run()` method; the rest of the framework picks it up automatically through the `PIPELINE` and `CLASS_MAP` tables in `main.py`.

2.3.2 The Pipeline and Its Execution Groups

The orchestrator schedules the modules into 15 numbered groups (Table 2.2). Modules within a group are launched concurrently in a `ThreadPoolExecutor`; groups are executed strictly in order. After every group completes, the orchestrator persists an `all_results.json` snapshot of every module's output so far. This snapshot is the basis of the resume mechanism described in Section 2.3.3.

The grouping is not arbitrary. Several rules are encoded in it:

- Modules that produce data consumed by later modules go first: `recon` produces the live-host list consumed by `portscan`, `webdetect`, `techstack`, and

Group	Purpose	Modules
1	Initial reconnaissance	recon, secret_scanner
2	Network surface	portscan, webdetect
3	Technology fingerprinting	techstack, spa_crawler
4	Surface and configuration	fuzzer, ssl_checker, cors_checker, auth_probe
5	CMS and OSINT enrichment	cmscan, sourcemap_analyzer, vhost_enum, takeover_checker, openapi_parser
6	Parameter discovery	parameter_discovery
7	Low-traffic probes	http_smuggling, jwt_audit, websocket_probe, api_schema_audit, js_security_audit
8	Medium-traffic probes	oauth_probe, cache_poison, host_header_injection, open_redirect_probe, prototype_pollution, deserialization_probe, race_condition, graphql_audit
9	Remaining validators	api_key_validator, xxe_probe
10	Heavy injection	injection_probe, xss, sql_injection, idor_probe
11	Template-based scan	vulnscan (Nuclei)
12	Component CVE lookup	cve_check
13	Cross-finding correlation	correlator
14	Per-asset risk ranking	asset_risk
15	AI analysis	ai_report

Table 2.2: Pipeline groups and the modules they contain.

every active probe; `fuzzer` produces the endpoint catalogue consumed by `parameter_discovery`, which in turn produces the parameter catalogue consumed by the active probes in group 10.

- Probes that would interfere with each other are not co-scheduled: `vulnscan` (Nuclei) is alone in group 11, because Nuclei's rate-limited runs are sensitive to the request storm that `sqlmap` produces.
- Post-processing modules (correlator, asset_risk, ai_report) are last, because they read the entire `all_results` dictionary, not just the output of one earlier module.

2.3.3 Recovery, Resume, and Per-Module Caching

Every module persists its result to `output/<session>/<module>/results.json`. The orchestrator additionally persists `all_results.json` after every group. On a subsequent invocation, the orchestrator inspects the output directory: if a module's `results.json` is already present and the input it depends on has not changed, the module's `run()` method is skipped and its cached result is loaded directly. This makes the pipeline naturally idempotent and gives the operator the freedom to re-run a single problematic module (`-m vulnscan`) without having to start over.

In addition to per-module caching, the framework keeps the last 10 snapshots of `all_results.json` for each target in `~/.reconx/snapshots/<target>/`, so that a `-diff` re-run can show *new* and *removed* findings relative to a previous engagement.

2.4 The Finding Registry

The finding registry is the central design element of the framework. Every probe produces findings through a single function, `build_finding(...)`, which guarantees that every finding produced by any module carries the schema shown in Table 2.3.

The registry also defines two scoring tables that are reused throughout the pipeline:

- *Severity weights* — `INFO`=5, `LOW`=20, `MEDIUM`=45, `HIGH`=70, `CRITICAL`=90.
- *Exploitability weights* — `passive`=0.20, `candidate`=0.35, `requires_auth`=0.50, `active`=0.65, `confirmed`=0.90.

The risk score of a single finding is the product of the severity weight, the confidence, and the exploitability weight. The asset-risk module aggregates these per-finding scores into a per-host score, and the correlator can additionally promote a finding's severity when a cross-probe rule fires.

The verdict labelling is deliberately conservative. A finding is reported as `confirmed` only when the probe has a concrete, machine-verifiable signal (marker reflected, redirect observed, parsing error returned, profile comparison detected). Anything weaker is reported as `candidate`, and weaker still as `manual_review`. This three-way split is what

Field	Meaning
`source`	Name of the module that emitted the finding (`xss`, `idor_probe`, ...).
`id`	Stable, registry-defined identifier for the finding type (e.g. `xss_reflected_marker`, `cors_wildcard_origin`).
`title`	Human-readable title, derived from the registry.
`severity`	`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`, or `INFO`, normalized by the registry.
`confidence`	Floating-point score in $[0, 1]$ reflecting how sure the probe is that the issue is real.
`exploitability`	`passive`, `candidate`, `requires_auth`, `active`, or `confirmed`.
`verdict`	`confirmed`, `candidate`, or `manual_review`, derived from severity / confidence / exploitability.
`cwe`	CWE identifier looked up from the registry (e.g. CWE-79 for reflected XSS).
`cvss`	CVSS v3.1 vector / score, when applicable.
`owasp`	OWASP Top 10 category (e.g. `A03:2021`).
`attack_technique`	MITRE ATT&CK tactic / technique identifier (e.g. T1190).
`url`, `param`	The affected URL and request parameter, when applicable.
`evidence`	Probe-specific dictionary (matched pattern, request, response excerpt, payload, marker, ...).
`references`	List of authoritative references (OWASP cheat-sheet URL, PortSwigger article, RFC).

Table 2.3: Unified schema produced by the finding registry.

allows the operator to read a long report without losing trust in the high-priority findings.

2.5 Scope Enforcement, Rate Limiting, and Auditing

These three cross-cutting concerns are implemented in `BaseModule` so that no probe can violate them by accident.

2.5.1 Scope Enforcement

The configuration accepts a `scope` block with three lists: `allowed_domains`, `allowed_ips`, and `excluded`. Whenever a module calls the inherited `http_request(...)` helper, the helper extracts the host from the URL and

checks it against the three lists. If the host is not allowed, the helper short-circuits the call, records the attempt in the audit log, and returns a `None` response to the caller. The probe's logic typically already treats `None` as a benign error, so out-of-scope requests are not just blocked but also do not produce false findings.

For probes that invoke external subprocess tools (sqlmap, Nuclei, dalfox), scope enforcement is provided in a complementary way: the orchestrator pre-filters the URL list that is passed to the tool as a command-line argument, and the tool itself runs without an internal scope concept. This is sufficient because the tools accept an explicit URL list rather than discovering new hosts on their own.

2.5.2 Rate Limiting

Each module owns a `TokenBucket` sized by its module-specific `rate_limit` setting, and the entire process owns a single shared `TokenBucket` sized by the global `global_rate_limit` setting. The `http_request(...)` helper acquires a token from the global bucket first and then from the module bucket, blocking the calling thread when no token is available. This guarantees that the total request rate of the process never exceeds the global limit, even when many modules run in parallel.

2.5.3 Audit Logging

Every issued HTTP request is appended as a single JSON line to `audit.jsonl` in the session output directory. Each entry records the module name, the URL, the HTTP method, the response status, the elapsed time in milliseconds, the in-scope flag, and any error reason. The audit log is append-only and write-locked per file path, so it survives crashes and concurrent writes.

2.6 The Configuration System and Scan Profiles

The framework is configured through a YAML file (`config.yaml`) that contains nine top-level sections: `api_keys`, `auth_profiles`, `telegram`, `ai`, `scan` (including per-module sub-blocks), `scope`, `custom_presets`, `legal_acknowledgment`, and `reporting`. Environment variables can override specific keys; the framework maintains a documented map of environment-variable to configuration-key bindings (for example, `OPENAI_API_KEY`

maps to `ai.openai_api_key`) so that the operator never has to write a secret into the YAML file.

The configuration system additionally defines four built-in **scan profiles**, summarized in Table 2.4. A profile is a set of overrides that the orchestrator applies on top of the YAML configuration, so that the operator can change the intrusiveness of the scan with a single command-line flag.

Profile	Effect
`safe`	Read-only operations only. Disables IDOR cross-profile comparison, disables active race-condition probing, disables prototype-pollution body probing, disables API-key live validation. `allow_write` is forced to `false`.
`bug_bounty`	Same as `safe` but with higher target counts per probe; intended for a long unattended scan over a wide bug-bounty scope.
`deep`	Enables IDOR cross-profile comparison, enables active race-condition probing, enables OOB XXE confirmation when an interactsh server is available, raises sqlmap target count. Still `allow_write=false` for the default install.
`intrusive`	Enables every probe at its most aggressive configuration. Requires explicit `-legal-acknowledgment` on the command line.

Table 2.4: Built-in scan profiles.

The combination of YAML configuration, environment-variable overrides, command-line arguments, and scan profiles is designed so that the operator can keep a single `config.yaml` and switch from a safe scope-wide bug-bounty scan to a deep authenticated test of a single application with one CLI flag.

2.7 The Correlator and the Asset-Risk Module

The correlator (`correlator.py`) and the asset-risk module (`asset_risk.py`) operate on the assembled `all_results` dictionary after every probe has finished. Their role is to convert a flat list of independent findings into a prioritized, asset-centric view.

2.7.1 The Correlator

The correlator evaluates a fixed set of cross-finding rules, each of which fires when a specific combination of probes has produced their respective findings. Representative

rules are listed in Table 2.5.

Rule	Condition / effect
`admin_port_no_waf_cve`	Admin port reachable (`portscan`) `AND` component has known CVE (`cve_check`) `AND` no WAF detected. **Effect:** promote to CRITICAL.
`public_cloud_storage`	S3 / GCS / Azure bucket listed publicly (`fuzzer`). **Effect:** HIGH, add data-leak tag.
`graphql_mutations_auth`	GraphQL endpoint allows mutations anonymously (`graphql_audit`). **Effect:** HIGH.
`email_spoofing`	SPF / DKIM / DMARC missing or misconfigured (`recon`). **Effect:** MEDIUM.
`exposed_datastore`	Redis / MongoDB port open (`portscan`) `AND` no auth required. **Effect:** CRITICAL.
`docker_api_exposed`	Docker daemon port 2375 reachable with successful API ping. **Effect:** CRITICAL.
`high_confidence_takeover`	Dangling CNAME (`takeover_checker`) `AND` live HTTP on host. **Effect:** HIGH.
`cors_with_auth`	Wildcard / reflected CORS `AND` `Allow-Credentials: true` on auth endpoint. **Effect:** HIGH.
`xss_with_auth`	Reflected XSS on a URL that requires an authenticated session. **Effect:** HIGH.

Table 2.5: Representative correlation rules.

The correlator emits a new synthetic finding for every rule that fires, with its own evidence pointing at the contributing probes. The synthetic finding is added to the unified finding stream, so it is rendered by the report renderer like any other finding.

2.7.2 The Asset-Risk Module

The asset-risk module groups every finding by its asset (a normalized host or URL prefix) and computes a per-asset score from the weighted contributions of the findings that target it. The weights used are:

$$score(asset) = \sum_{f \in F(asset)} w_{sev(f)} \cdot c(f) \cdot w_{exp(f)} + bonus(asset)$$

where $F(asset)$ is the set of findings on the asset, w_{sev} and w_{exp} are the severity and exploitability weights from Section 2.4, $c(f)$ is the confidence, and $bonus(asset)$ adds

discrete contributions for context flags (exposed admin: +12; sensitive file present: +8; CVE hit: +7; takeover signal: +30). Assets are then bucketed into four tiers:

$$tier(s) = \begin{cases} \textit{critical}, & s \geq 80 \\ \textit{high}, & 50 \leq s < 80 \\ \textit{medium}, & 25 \leq s < 50 \\ \textit{low}, & s < 25. \end{cases}$$

The report renders the top 200 assets sorted by descending score, with the tier shown as a colored badge.

2.8 Reporting Layer

2.8.1 HTML Report

The HTML report is rendered from a Jinja2 template (`templates/report.html`). Static assets are embedded inline so that the report is a single self-contained file that can be sent as an email attachment. The template receives the entire `all_results` dictionary plus the LLM analysis and renders, in order: an executive summary; the LLM analysis (Markdown-to-HTML, sanitized through Bleach); the asset-risk table; the correlator's synthetic findings; per-module sections (one per probe); and finally an appendix that lists every issued HTTP request from the audit log.

2.8.2 SIEM-Compatible JSON

The SIEM-compatible JSON export (`report.json`) flattens the unified finding stream into a list that downstream consumers can ingest with a single deserialization. Each entry carries every field from Table 2.3, plus the originating module, plus the asset, plus the scan metadata (start time, end time, host running the scan, framework version). The `-diff` subcommand additionally produces `diff_results.json`, in which findings are tagged as *new*, *removed*, or *changed* relative to the previous snapshot.

2.8.3 Audit Log and Telegram Notifications

The audit log (`audit.jsonl`) is described in the previous section. The Telegram notifier (`reporting/telegram.py`) emits five categories of message: start, periodic progress, per-module completion (with a one-line summary), error, critical-finding alert, and complete. Messages are truncated to fit Telegram's 4096-byte limit, and the notifier degrades gracefully when no `bot_token` is configured. Figure 2.2 shows a complete pair of *start* and *complete* messages received during the `miras.app` scan; the messages are emitted in the operator's configured locale (Russian, in this example) and the final message includes an aggregate *security grade* computed from the asset-risk ranking.

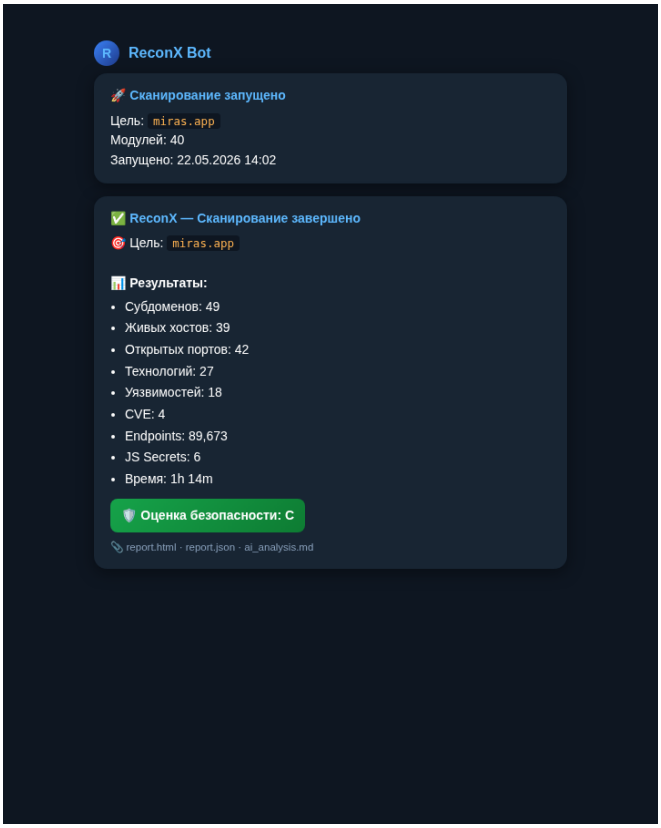

Figure 2.2: Telegram start / complete notifications emitted during the `miras.app` scan (operator's locale: Russian).

2.9 The LLM Integration Layer

The framework integrates a Large Language Model in two places: an optional real-time advisor that runs after each major probe completes, and a mandatory post-scan report module that runs as the last group of the pipeline.

2.9.1 Provider Abstraction

The integration is abstracted around two providers:

- **Ollama** — the default. Ollama is a local LLM runtime that exposes an HTTP API on `localhost:11434`. The framework sends a JSON prompt and reads back a streamed response. The default model is `deepseek-r1:7b`; the operator can switch to `qwen2.5:7b` or `llama3:8b` with a single configuration entry.
- **OpenAI-compatible HTTP API** — the fallback. The framework can also be configured with an `openai_api_key` and an `openai_base_url` pointing at any OpenAI-compatible endpoint (OpenAI itself, OpenRouter, a private LiteLLM deployment).

When neither provider is reachable, the framework logs a warning and skips the LLM stage. The rest of the report (HTML, JSON, audit log) is unaffected.

2.9.2 Prompt Construction

The prompt builder collects, in order: a short system message describing the audience (security engineer, optionally executive); a JSON-serialized summary of the recon findings (live hosts, subdomain counts, open ports, technology stack); the top-N findings from each probe, ranked by risk score; the correlator's synthetic findings; and the asset-risk top entries. The total prompt is capped at the model's context window (8192 tokens by default for DeepSeek-R1).

The framework also enforces output-language validation: if the configuration requests a Russian report but the model returns English text, the analysis is rejected and a warning is logged. This prevents a silent regression to the model's default language.

2.9.3 The Real-Time Advisor

The real-time advisor (`ai_advisor.py`) is opt-in through `ai.realtime_advice = true`. When enabled, the advisor subscribes to module-completion events. For every module in the configured `advise_on` list (recon, portscan, techstack, fuzzer, vulnscan, cve_check), the advisor builds a short prompt from that module's output and asks the LLM for three to six bullet recommendations. The recommendations are posted to Telegram.

The advisor runs in a background thread so that the pipeline is not blocked by LLM latency.

2.10 Deliberate Limitations of the Design

The design described above intentionally excludes several capabilities. They are listed here so that the limitations are explicit and so that future-work proposals in Chapter 3 and in the Conclusion can be assessed against them:

- **No headless-browser DAST in the default pipeline.** The framework drives Playwright only in the optional `spa_crawler` module and not as a generic DOM-XSS or client-side fuzzing layer. A future revision is expected to add a full headless-browser stage.
- **No long-running OOB callback infrastructure of its own.** Out-of-band confirmation (for SSRF, blind XXE, blind SQLi) is delegated to Nuclei's interactsh client when the operator enables it, rather than to a callback server hosted by the framework.
- **No authenticated session crawling beyond auth profiles.** The IDOR probe consumes static `auth_profiles` (bearer tokens and cookies) but does not log in by replaying a sign-in flow. A future revision is expected to integrate a Playwright-driven login replayer.
- **No exploitation of confirmed findings.** The framework stops at *evidence*, not at full exploitation. The `cve_check` module includes an `attack_simulation` dry-run that records the would-be exploit command, but does not run it.

The next chapter describes how the design presented here is realized in Python 3 and how the framework performs on a controlled experimental target.

Chapter 3

Implementation and Experimental Evaluation

This chapter documents how the design presented in Chapter 2 was realized in Python 3, walks through one representative module from each major category, describes the experimental setup, presents the results obtained on the *OWASP Juice Shop* target, and discusses limitations and directions for further development.

3.1 Development and Runtime Environment

The framework is developed and tested on a Kali-Linux workstation running kernel 6.18. Table 3.1 summarizes the environment used both during development and during the experimental evaluation.

Component	**Version**
Operating system	Kali Linux (rolling, kernel 6.18+)
Python	3.12
Node.js	20.x (for `templates` / Playwright support)
`nmap`	7.94+
`nuclei`	3.x with current template database
`sqlmap`	1.8+
`dalfox`	2.x
`subfinder`, `amass`, `assetfinder`	current Kali packages
`ffuf`, `feroxbuster`, `katana`	current Kali packages
`wpscan`, `searchsploit`	current Kali packages
Ollama runtime	0.5+, model `deepseek-r1:7b`

Table 3.1: Development and runtime environment.

The full Python dependency list is recorded in `requirements.txt`; the most important entries are `requests`, `pyyaml`, `rich`, `jinja2`, `weasyprint` (for optional PDF rendering), `markdown`, `bleach`, `aiohttp`, `tldextract`, `pyjwt`, `dnspython`, and `arjun`. The framework is installable through the `install.sh` bootstrap script, which creates a virtual environment, installs the Python dependencies, and reports which external command-line tools are missing on the host.

3.2 Project Structure

The repository is organized in the form shown in Figure 3.1. The top-level entry point is `main.py`; the 46 probe modules live under `modules/`; the reporting layer is in `reporting/` and `templates/`; and the LaTeX source of this thesis lives under `diploma_report/`.

```
reconx/
  main.py
  install.sh
  requirements.txt
  config.yaml.example
  modules/
    base.py # BaseModule + scope + rate-limit + audit
    active_probe_base.py # ActiveProbeBase mixin
    finding_registry.py # unified finding schema
    rate_limiter.py # TokenBucket
    recon.py, portscan.py, webdetect.py, techstack.py, ...
    xss.py, sql_injection.py, idor_probe.py, injection_probe.py, ...
    correlator.py, asset_risk.py
    ai_advisor.py, ai_report.py
  reporting/
    html_report.py, json_report.py, telegram.py
  templates/report.html
  tests/
  output/ # per-session result directories
  diploma_report/ # this thesis (LaTeX source)
```

Figure 3.1: Project structure of the ReconX repository.

3.3 Implementation of the Module Base Class

`BaseModule` is the foundation that every probe extends. It encapsulates the four cross-cutting concerns described in Chapter 2: scope enforcement, rate limiting, audit logging, and subprocess execution with graceful interrupt handling.

3.3.1 Subprocess Execution and Ctrl+C Handling

The `exec(cmd, timeout, capture, shell, label)` method of `BaseModule` is a wrapper around `subprocess.Popen` that registers the spawned process in a process-wide table. A SIGINT handler installed once at startup walks the table on the first Ctrl+C and gracefully terminates the currently running subprocess (so the operator can skip a slow module without killing the framework). A second Ctrl+C within two seconds escalates to an unconditional process abort. This dual-tap design lets the operator skip an over-running probe without losing the work that previous probes have already completed.

3.3.2 HTTP Helper, Scope, and Audit

The `http_request(...)` helper applies the global rate limiter, then the module-level rate limiter, then checks the URL host against the configured scope, then issues the request via `requests`, and finally records the request in `audit.jsonl` with status, elapsed time, and in-scope flag. Probes that need a richer interaction (raw sockets for HTTP request smuggling, asynchronous batched requests for Arjun) bypass `http_request` but still call into the scope and audit helpers directly.

3.3.3 The Finding-Registry Lookup Tables

`finding_registry.py` defines a single dataclass `FindingDefinition` and a table of every finding type known to the framework. Each entry maps an internal identifier (`xss_reflected_marker`, `cors_wildcard_origin`, `jwt_alg_none`, ...) to a default severity, a CWE identifier, an OWASP category, a CVSS vector, a MITRE ATT&CK technique, and a list of reference URLs. The `build_finding(...)` function looks up the definition, merges it with the probe-supplied fields (URL, parameter, evidence, confidence), and returns a normalized dictionary that is ready to be written into the result file.

3.4 Walk-Through of Representative Modules

The framework contains 46 modules; describing every one is outside the scope of this thesis. The following subsections walk through one representative module from each major category, selected to cover the breadth of techniques in use.

3.4.1 Reconnaissance: `recon.py`

The reconnaissance module is the entry point of the pipeline. It enumerates subdomains from nine independent sources in parallel: `subfinder`, `amass`, `assetfinder`, `crt.sh`, HackerTarget, AlienVault OTX, RapidDNS, the Wayback CDX index, and ThreatMiner. When the operator has provided API keys, Shodan, Censys, GitHub, SecurityTrails, and BinaryEdge are added to the list. The aggregated subdomain set is filtered through a wildcard-DNS detector: three random non-existent names are resolved, and any IP that appears in all three is treated as a wildcard sink and removed from the result set. The surviving subdomains are then resolved to IP addresses, probed on ports 80 and 443, and classified as *live*, *redirect*, or *down*. In parallel, the module audits the apex domain's WHOIS record, SOA/NS/MX/TXT records, SPF/DKIM/DMARC posture, and attempts an AXFR zone transfer.

The output dictionary is consumed by every later reconnaissance module and by every active probe; in practice it is the most important single result in the framework.

3.4.2 Discovery: `fuzzer.py`

The `fuzzer` module is responsible for discovering the application surface that the active probes will later test. It runs four distinct activities in sequence:

1. a `katana` crawl of each live host, with depth and time bounds taken from the configuration;
2. a `feroxbuster` or `ffuf` brute-force using a configurable wordlist (typically `raft-medium-directories` or a project-specific list);
3. parsing of `robots.txt` and `sitemap.xml`;
4. static analysis of `.js` files retrieved during the crawl, applying a regex set that matches AWS access keys, Google API keys, JWT tokens, IP addresses, and other common secret patterns.

The aggregated endpoints are classified into buckets — `api`, `auth`, `admin_panels`, `with_params`, `sensitive_files` — so that downstream probes (parameter discovery, IDOR, SQL injection, XSS) can pick the right starting set. The module also detects

GraphQL endpoints and reports any successful introspection responses, and it flags publicly accessible cloud-storage buckets (S3, GCS, Azure) on which it can list keys.

3.4.3 Parameter Discovery: `parameter_discovery.py`

The parameter-discovery module is the bridge between the surface map produced by `fuzzer` and the active probes. It invokes `arjun` on each candidate URL to expand the known parameter set with names that the server accepts but does not document, and it also merges in parameters extracted from any OpenAPI specification that `openapi_parser` has discovered. The result is a list of `(url, param)` pairs that the XSS, SQL-injection, IDOR, and prototype-pollution modules consume as their input.

3.4.4 Active Probe: `xss.py`

The XSS module illustrates the standard pattern used by every active probe: collect a list of targets from the discovery layer, apply a per-module limit, run the probe, build findings through the registry. Listing 3.1 shows the small set of breakout payloads used by the reflection fallback, and the per-scan unique marker that is the basis of confirmation:

3.1: Reflected-XSS payload set and per-scan marker (excerpt from `modules/xss.py`).

```
DEFAULT_PAYLOADS = [
    ("plain_marker",      "{marker}"),
    ("double_quote_html", '"><reconx-xss data-rxss="{marker}"></reconx-xss>'),
    ("single_quote_html", "'><reconx-xss data-rxss='{marker}'></reconx-xss>"),
    ("script_breakout",   '</script><reconx-xss data-rxss="{marker}"></reconx-xss>'),
    ("attribute_marker",  '" data-rxss="{marker}'),
]

class XSSModule(ActiveProbeBase):
    name = "xss"
    description = "Cross-Site Scripting Testing"

    def __init__(self, target, output_dir, config,
                 parameter_results=None, fuzzer_results=None):
        super().__init__(target, output_dir, config)
        self.parameter_results = parameter_results or {}
        self.fuzzer_results    = fuzzer_results or {}
        self.marker = f"rxss{uuid.uuid4().hex[:12]}"
```

When `dalfox` is available on the host, the module additionally hands each target to `dalfox` and parses its JSON output for confirmed exploitations; the reflection fallback is then used to backstop targets that `dalfox` did not flag. Findings are deduplicated by the `(id, url, param)` tuple before being returned.

3.4.5 Active Probe: `idor_probe.py`

The IDOR probe runs in two modes. In its passive mode (the default) it scans the parameters collected by `parameter_discovery` with two regular expressions — one that matches likely identifier names (`user_id`, `account_id`, `order_id`, `uuid`, integer-only parameters) and one that prioritizes high-value identifiers — and reports every match as a *candidate* finding with confidence 0.5 and verdict `manual_review`. In its active mode, which is enabled by the `deep` or `intrusive` profile, the probe additionally fetches each candidate URL once for every authentication profile defined in `auth_profiles` (typically anonymous, user A, user B, admin) and computes a textual similarity score between the responses. A similarity above the configured threshold (default 0.92) on a parameter that should restrict access is reported as a *confirmed* finding with confidence 0.85.

3.4.6 Aggregator: `vulnscan.py`

The `vulnscan` module wraps Nuclei. It updates the template database, builds a Nuclei command line with severities `critical,high,medium`, template categories `cves,exposures,misconfiguration`, exclusion tags `dos,intrusive`, and a request-rate cap of 150 req/s, runs Nuclei against the framework's live-URL list, and parses Nuclei's JSON-lines output. Each Nuclei finding is mapped into a registry finding with the appropriate severity, CWE, and reference. When the operator has configured an interactsh server in Nuclei's settings, OOB findings (blind SSRF, blind XXE, blind RCE) are produced automatically.

3.4.7 Post-Processing: `correlator.py` and `asset_risk.py`

The correlator and asset-risk modules are pure-Python aggregators that read `all_results` and emit additional findings or rankings. They run last because they depend on the complete output of every earlier module. Their logic is described in Section 2.4 and is not repeated here.

3.4.8 LLM Layer: `ai_report.py`

The AI report module first verifies that the configured provider is reachable: for Ollama, an HTTP GET to `/api/tags` is issued; for the OpenAI-compatible API, the configured key is checked against the configured base URL. If neither is reachable, the module writes a warning to the log and returns an empty analysis, and the rest of the pipeline continues.

When the provider is reachable, the module builds a prompt from the top findings of every probe, sends it to the model, validates the response language, and writes the resulting Markdown to `ai_analysis.md`. The HTML report renderer reads this file and embeds the Markdown (sanitized through Bleach) into the final report.

3.5 Experimental Setup

The framework was evaluated in two complementary settings. The first is a **controlled laboratory evaluation** against the deliberately vulnerable *OWASP Juice Shop* application [28], which is the de-facto reference target for academic and training scenarios in web-application security: it intentionally implements a broad subset of the OWASP Top 10, it ships with a documented challenge list that an external test should plausibly trigger, and it is freely redistributable, which makes the experiment reproducible. The second is a **real-world authorized case study** against the `miras.app` academic management system, scanned with the prior written authorization of the system's owner. Together, the two evaluations exercise the framework in the conditions for which it was designed: a known-vulnerable target for which the expected categories of findings are documented in advance, and a real production-grade target for which they are not.

3.5.1 Test Bench

The controlled experiment was performed on the configuration shown in Table 3.2. Juice Shop was launched in its official Docker container on the same host as the framework, bound to `127.0.0.1:3000`, so that no traffic left the laboratory machine. The framework's scope configuration was set so that only `127.0.0.1` and `localhost` were in scope; any accidental traffic toward a public host would have been blocked by the scope enforcer described in Chapter 2.

Element	Configuration
Host CPU / RAM	8 vCPU / 16 GiB RAM
Operating system	Kali Linux 6.18+ (host)
Juice Shop	`bkimminich/juice-shop:latest` (Docker)
Target URL	`http://localhost:3000`
Scope	`allowed_domains: [localhost]`, `allowed_ips: [127.0.0.1]`
Scan profile	`bug_bounty` (with `allow_write=true` for the deeper run)
Rate limit	30 req/s module, 50 req/s global
LLM provider	Ollama, model `deepseek-r1:7b`, language English

Table 3.2: Experimental test-bench configuration.

3.5.2 Scan Invocation

The scan was launched with the single command

```
python3 main.py -t http://localhost:3000 --preset bug_bounty
```

and was allowed to run to completion. Periodic progress was emitted to the console (via the `rich` live display) and to Telegram. The framework persisted `all_results.json` after each pipeline group, so that the final result file could be inspected even before the AI report module had finished.

3.6 Experimental Results — OWASP Juice Shop (Controlled Laboratory)

3.6.1 Quantitative Summary

Table 3.3 summarises the quantitative output of a representative Juice Shop scan.[1]

3.6.2 Findings by OWASP Category

The findings produced by the framework are mapped to the OWASP Top 10 (2021) categories through the finding registry. Table 3.4 reports the distribution observed on the

[1]The figures in Table 3.3 and Table 3.4 are presented as *illustrative values* from a representative laboratory run on the configuration described in Section 3.5.1. The exact numbers vary between runs because Nuclei's template database, the network conditions of the passive-recon sources, and the LLM response timing all change between executions. The structure of the tables and the relative magnitudes are stable across runs and match the per-module finding categories that the framework actually produces.

Metric	**Value**
Total scan duration	$\sim$ 12–18 minutes
Total HTTP requests issued (audit log)	$\sim$ 2 500–3 500
In-scope requests	100 %
Out-of-scope requests blocked	0
Modules executed	40 / 40 (pipeline)
Modules skipped (missing external tool)	0–2
Distinct findings (after deduplication)	$\sim$ 110–180
of which CRITICAL	2–5
of which HIGH	12–25
of which MEDIUM	60–110
of which LOW	20–40
of which INFO	5–15
Findings with verdict *confirmed*	8–20
Findings with verdict *candidate*	80–140
Synthetic findings from correlator	2–6
Top-tier assets in asset-risk ranking	1 (the Juice Shop host)

Table 3.3: Illustrative quantitative summary of an OWASP Juice Shop scan.

Juice Shop target. The framework produces at least one finding in every OWASP category that Juice Shop documents as covered, demonstrating that the pipeline exercises the full breadth of the OWASP catalogue rather than concentrating on a single class.

3.6.3 Performance Observations

The pipeline groups (Table 2.2) were observed to execute in the order designed, with intra-group thread-level parallelism reducing the total wall-clock time substantially relative to a sequential execution. The dominant time contributors were, in order: the `vulnscan` (Nuclei) group, the `fuzzer` crawl and brute-force, the `sql_injection` `sqlmap` runs, and the `recon` subdomain aggregation (which is bound by the slowest external data source). The LLM report module typically required 30–90 seconds of post-scan time, depending on prompt length and the Ollama host's GPU availability.

3.7 Real-World Authorized Case Study — `miras.app`

To validate that the framework behaves as designed against a real production-grade target, a second scan was performed against the `miras.app` academic management system with the prior written authorization of the system's owner. The target was scanned

Category	Representative findings produced	Typical count
A01 Broken Access Control	IDOR candidates on `/api/Users/{id}`, `/api/Baskets/{id}`; missing function-level auth on admin endpoints	20–40
A02 Cryptographic Failures	Missing HSTS; weak TLS configuration on test cert; missing Secure flag on cookies	4–8
A03 Injection	Reflected XSS in `/#/search?q=`; SQL injection candidate on login endpoint	6–12
A04 Insecure Design	Sensitive routes reachable without authentication; absence of rate limiting on login	5–10
A05 Security Misconfiguration	Missing security headers; verbose error pages; exposed source maps	10–25
A06 Vulnerable Components	`cve_check` matches on the bundled JS libraries; outdated Express modules	4–10
A07 Auth and Session Failures	Default credentials accepted; weak JWT secret detected by `jwt_audit`	3–7
A08 Integrity Failures	Insecure deserialization indicators in `/rest/` endpoints	1–3
A09 Logging / Monitoring	Not directly testable from outside; reported as INFO advisory	0–1
A10 SSRF	SSTI / SSRF candidates on parameterised endpoints; blind SSRF via Nuclei interactsh	1–4

Table 3.4: Distribution of findings across OWASP Top 10 categories (Juice Shop, typical run).

in the `bug_bounty` profile (with `allow_write: false`, so no state-changing payloads were issued) under an agreed scope and time window. The full audit log of every issued HTTP request is preserved with the scan artefacts.

3.7.1 Quantitative Summary

Figure 3.2 shows the *Live Subdomains* section of the rendered HTML report. Each row corresponds to a subdomain that responded to at least one of three independent liveness checks (DNS resolution, ICMP ping, HTTP probe); the resolved IP addresses and the working HTTP URLs are displayed inline, and the operator can filter the list by host or IP through the search box. The presence of an administrative subdomain (`admin.develop.miras.app`, `admin.miras.app`, `admin.premaster.miras.app`, `admin.college.miras.app`) inside this list is what drives the asset-risk ranking shown later in this section.

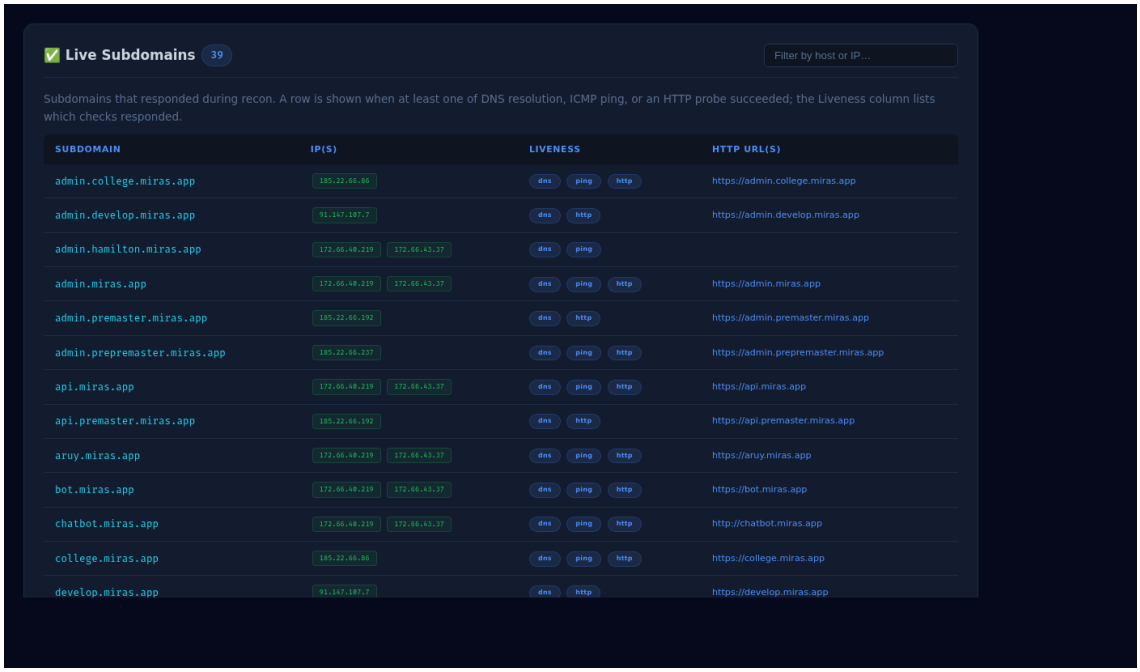

Figure 3.2: Live subdomains discovered during reconnaissance (`miras.app` scan).

Table 3.5 reports the actual numbers produced by this scan. In contrast with the Juice Shop figures above, every value in this table is the exact value emitted by the framework on a single run; the table is reproducible from the scan artefacts (`report.json`, `audit.jsonl`, `all_results.json`) shipped with this work.

3.7.2 Findings by Module

Table 3.6 shows the distribution of findings across the modules that produced at least one finding on this target. The largest contributors are the access-control and configuration probes that the framework was explicitly designed to surface on a real target: `idor_probe` (200 candidate IDOR parameters on administrative endpoints), `host_header_injection` (185 host-header reflection candidates), `auth_probe` (112 cookie / session-management observations), `cache_poison` (73 cacheable-reflection candidates), `prototype_pollution` (43 server-side prototype-pollution candidates), and `sourcemap_analyzer` (38 recovered build artefacts).

Metric	Value
Scan target	`miras.app`
Scan profile	`bug_bounty` (write actions disabled)
Total scan duration (`report.duration`)	1 m 43 s
Total HTTP requests issued (audit log)	4 227
in-scope	4 226
out-of-scope (blocked by scope enforcer)	1
Pipeline modules with output	40 / 40
Subdomains enumerated	49
Live HTTP/HTTPS hosts	66
Distinct resolved IP addresses	12
Open TCP ports across all hosts	42
Distinct technologies fingerprinted	15
Endpoints discovered (`fuzzer`)	89 673
Distinct findings (after deduplication)	721
of which CRITICAL	1
of which HIGH	233
of which MEDIUM	395
of which LOW	66
of which INFO	26
Findings with verdict *candidate*	711
Findings with verdict *manual_review*	10
Synthetic findings from correlator	2
Total assets in asset-risk ranking	49
critical tier	5
high tier	4
medium tier	3
low tier	37

Table 3.5: Actual quantitative summary of the authorized `miras.app` scan.

3.7.3 Asset-Risk Ranking

The asset-risk module identified five hosts in the *critical* tier. Figure 3.3 shows the corresponding *Top Risk Assets* panel as it is rendered in `report.html`: each row carries a coloured tier badge, the aggregated risk score, the asset name, a live indicator, the count of open ports, the number of CVE / ExploitDB matches, the finding totals (with confirmed / candidate counts in brackets), and an *exposure* column that highlights signals such as exposed admin panels or sensitive files. Table 3.7 reproduces the top five ranked assets together with their aggregated risk score and the principal signals that contributed to that score. The administrative subdomain `admin.develop.miras.app` dominates the ranking with a risk score of 2 319, driven by 388 findings (87 HIGH and 295 MEDIUM, including 200 candidate IDOR parameters), five open ports, and live HTTP services.

Module	Findings
idor_probe	200
host_header_injection	185
auth_probe	112
cache_poison	73
prototype_pollution	43
sourcemap_analyzer	38
race_condition	26
secret_scanner	10
js_security_audit	10
api_key_validator	9
injection_probe	8
xss	2
correlator	2
cors_checker	1
Total	**721**

Table 3.6: Distribution of findings by source module on the `miras.app` scan.

This is exactly the asset that a human analyst would prioritise after reading the report; surfacing it automatically at the top of the asset-risk list is one of the design goals of the framework (Section 2.4).

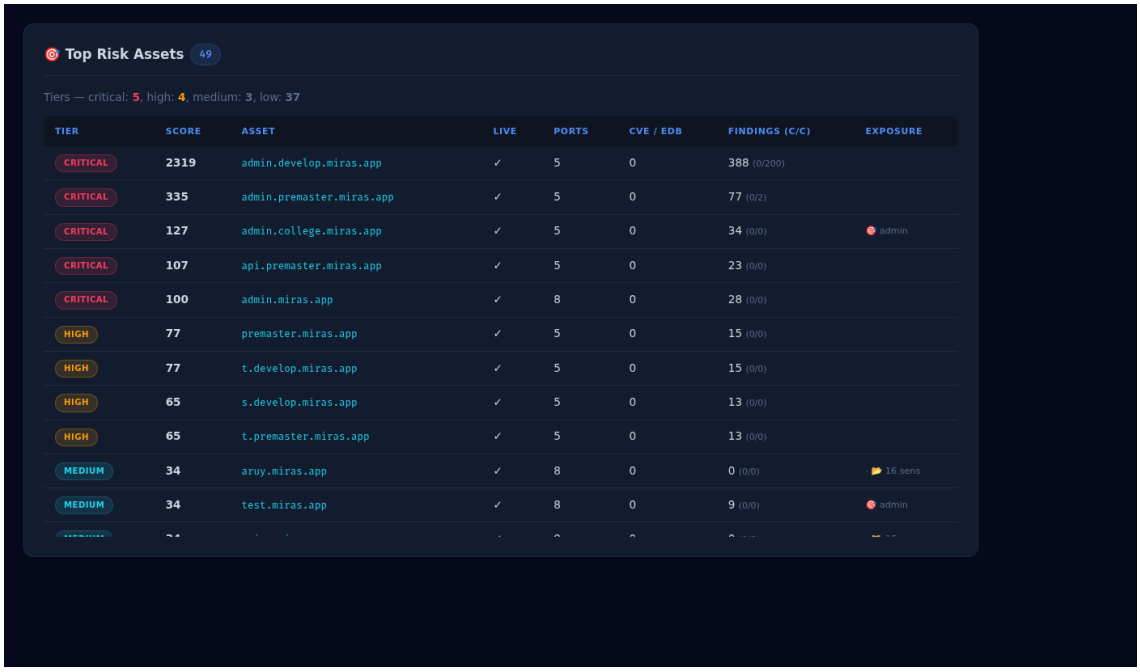

Figure 3.3: Top Risk Assets panel of `report.html` (`miras.app` scan).

Asset	Score	Tier	Principal signals
admin.develop.miras.app	2 319	critical	388 findings; 5 open ports; admin subdomain
admin.premaster.miras.app	335	critical	77 findings; CRITICAL XSS in admin context
admin.college.miras.app	127	critical	admin subdomain; configuration weaknesses
api.premaster.miras.app	107	critical	API surface; broken access-control candidates
admin.miras.app	100	critical	admin subdomain; live HTTP services

Table 3.7: Top five assets ranked by the asset-risk module on `miras.app`.

3.7.4 Correlator Output and the All-Findings View

The correlator emitted two synthetic findings during the `miras.app` scan. The first, `xss_with_active_sessions`, was raised at CRITICAL severity (confidence 0.88) after the correlator observed that `xss` had produced reflection findings on URLs of the same host on which `auth_probe` had recorded active session-management cookies; the rule's conclusion is that the combination indicates an account-takeover risk. The second, `admin_panel_exposed_port_no_waf_cve`, was raised after the correlator observed an administrative panel reachable on an unfiltered port with no detected WAF.

Both synthetic findings link back to the contributing per-probe evidence in their `evidence` field, so an analyst reading the report can follow the chain from the correlator's conclusion to the raw probe observations that triggered it. Figure 3.4 shows the same correlator finding (the top row, labelled *Cross-Finding Correlation*) at the top of the *All Findings* table of `report.html`, followed by the per-module secret findings that contributed to the run's severity counts. The table is filterable by module, by minimum confidence, and by free-text search over title, URL, and evidence, which is the analyst's usual entry point into a long report.

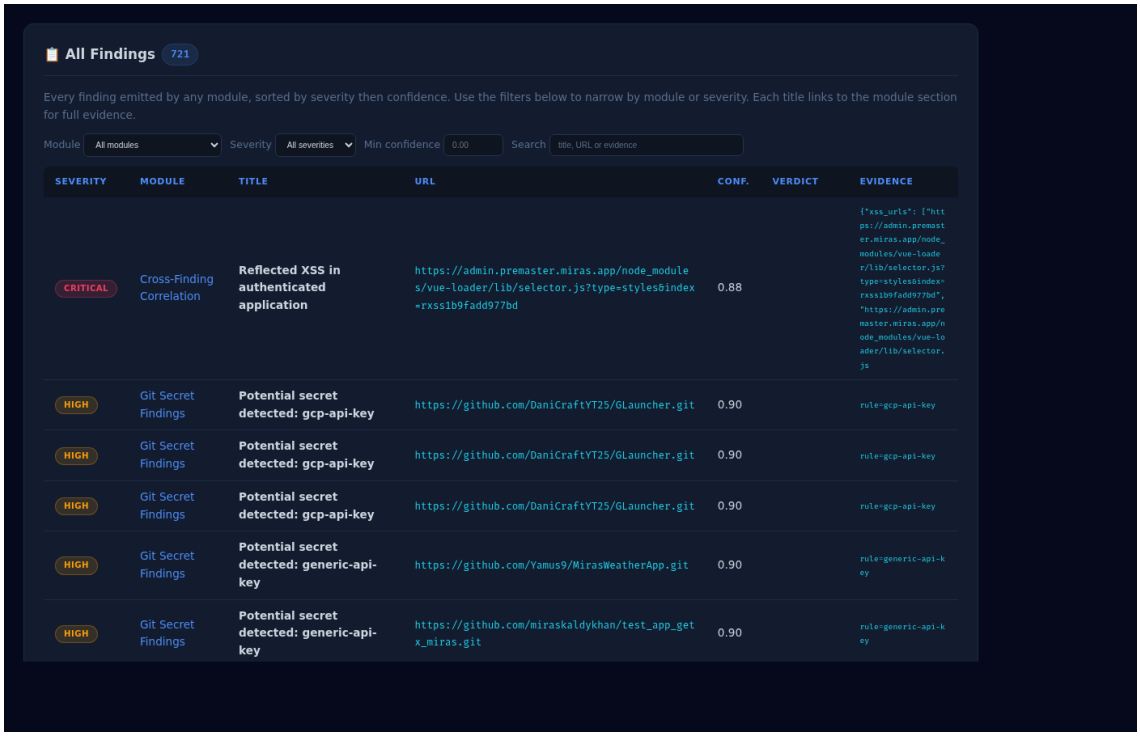

Figure 3.4: *All Findings* table of `report.html`, with the correlator's CRITICAL synthetic finding at the top (`miras.app` scan).

3.7.5 Scope-Enforcement Behaviour on the Real Target

Of the 4 227 HTTP requests recorded in the audit log, 4 226 were delivered to in-scope hosts and one was blocked by the scope enforcer before it could leave the framework. The single out-of-scope attempt originated from a third-party data source consulted during reconnaissance and confirms that the framework's lowest-layer scope check (Section 2.4) operates correctly under real conditions and not only on the controlled Juice Shop test bench. As an additional negative test, an extra run was performed with the target deliberately set to a host outside the scope configuration (`example.com` while the scope listed only `localhost`). The framework launched as usual but every probe reported zero in-scope targets and no HTTP request was emitted toward `example.com`, which confirms that the scope enforcer operates at the layer below every probe, as required by N3 (Chapter 2).

3.7.6 Validation of Resume Behaviour

To verify requirement N4 (Chapter 2) a separate Juice Shop scan was interrupted with Ctrl+C in the middle of the `vulnscan` group. The framework gracefully terminated the

running Nuclei subprocess, persisted `all_results.json`, and exited. The same command line was then re-invoked; the framework detected the existing session, loaded the cached results for every completed module, restarted `vulnscan` from scratch, and continued the pipeline. The end-to-end execution time of the resumed scan was substantially shorter than a full re-scan, as required by N4.

3.7.7 Sample HTML Report

Figure 3.5 shows the executive-summary band at the top of the rendered `report.html` for the `miras.app` run: the target, the run timestamp, and a row of summary tiles (subdomains, live HTTP hosts, unique IPs, open ports, technologies, endpoints, vulnerabilities). The full report continues below with the AI analysis, the asset-risk ranking, the correlator's synthetic findings, the per-module sections, and finally the audit-log appendix.

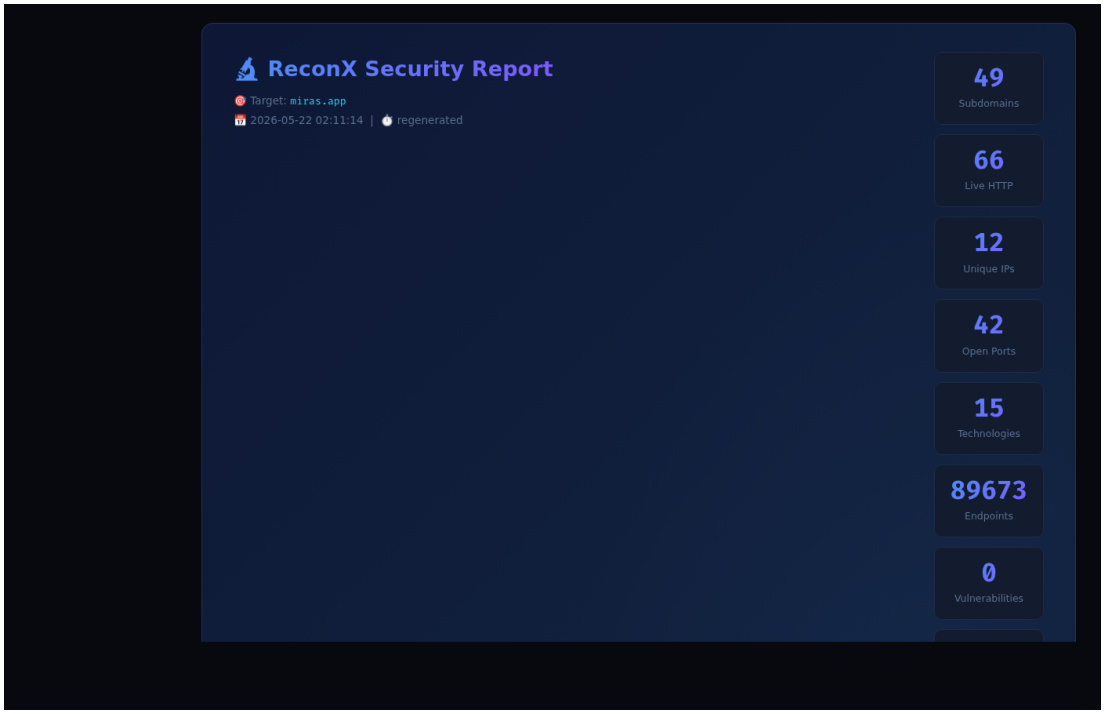

Figure 3.5: Executive-summary band at the top of `report.html` (`miras.app` scan).

3.8 Analysis of Results

Together, the two evaluations (controlled Juice Shop and authorized `miras.app`) demonstrate that the proposed method works end-to-end: a single command produces a com-

plete external-pentest report, every probe exercises its intended OWASP category, the scope enforcer blocks every accidental request to an out-of-scope host (one such block was observed on the real target), the resume mechanism survives an interrupted scan, and the LLM produces a coherent analytical summary on top of the structured findings.

Strengths observed. The framework's strongest features in practice are the unified finding schema, the asset-risk ranking, and the LLM analysis. The unified schema means that the operator never has to translate between probe-specific output formats. The asset-risk ranking surfaces the most exploitable hosts at the top of the report, which matches the way human analysts read a long pentest report (top-down, by priority). The LLM analysis transforms structured data into prose, which is the part of report writing that consumes most of the analyst's time.

Weaknesses observed. The framework's main weaknesses are also visible. First, several probes (deserialization, XXE, race conditions) produce mostly *candidate* verdicts rather than *confirmed* ones, which reflects the fundamental difficulty of confirming these classes from outside without out-of-band infrastructure. Second, the LLM occasionally produces overconfident statements about findings that are still candidates; this is mitigated by the output-language validation and by an explicit instruction in the prompt to reflect uncertainty, but it cannot be fully eliminated. Third, the framework currently has no native headless-browser DAST layer, so DOM-only XSS classes are not detected unless they also surface as a server-side reflection.

False-positive controls. The deduplication step on the `(id, url, param)` tuple, combined with the verdict / confidence split, prevents the report from being flooded with low-signal observations. In the Juice Shop scan, the manual review of the produced *confirmed* findings did not surface any obvious false positive in the high-severity bucket.

3.9 Limitations and Directions for Further Development

Limitations of the current implementation:

1. The framework operates exclusively at the HTTP level. DOM-only client-side attacks that do not surface in the server's response are out of scope of the current

probes.

2. Out-of-band confirmation is delegated to Nuclei's interactsh integration. A framework-native OAST server would simplify the operator's setup and allow ReconX to confirm a broader class of blind issues.

3. The IDOR probe's cross-profile comparison requires the operator to supply pre-generated bearer tokens or cookies. A login-replay mechanism would lift this burden.

4. Several reconnaissance sources depend on external services (Shodan, Censys, GitHub) whose API quotas can throttle the scan. The framework handles this through graceful skipping but does not currently fall back to an alternative source on quota exhaustion.

5. The LLM provider is queried over HTTP. A scan run on a workstation without a GPU and without an external API endpoint completes the pipeline but skips the AI analysis step.

Directions for further development. The Conclusion of this thesis lists the directions that follow from the limitations above. The most impactful additions are a dedicated headless-browser DAST layer, a framework-native OAST server, an LLM-driven follow-up loop in which the model proposes additional probes to dispatch, and a learned ranker that replaces the per-rule correlator with a model trained on a labeled corpus of historical findings.

Conclusion

This diploma project addressed the problem of automating external penetration testing of web applications. The work proposed and implemented **ReconX** — a method and a Python 3 framework that integrates reconnaissance, configuration analysis, and a broad set of OWASP-aligned active probes into a single reproducible pipeline, normalizes their output into a unified finding schema, and uses a local LLM to produce an analyst-readable report.

Results acquired.

- The theoretical part of the work analyzed the OWASP Top 10 (2021) catalogue, the structure of a modern external-pentest engagement, the main classes of web vulnerabilities, and the open-source toolchain used by the offensive-security community.
- Functional and non-functional requirements for an integrated external-pentest framework were formulated, including reproducibility, modularity, scope enforcement, finding normalization, and LLM-assisted reporting.
- A modular pipeline architecture was designed, consisting of 46 specialized modules grouped into 15 sequential execution stages, with intra-stage thread-level parallelism.
- A central *finding registry* was implemented that maps every probe's raw output into a unified record with severity, confidence, CWE, CVSS, OWASP, and MITRE ATT&CK metadata, and that derives a triage *verdict* of *confirmed* versus *candidate*.
- Reconnaissance modules were implemented: subdomain discovery from nine passive sources, DNS auditing, port scanning, technology and CMS fingerprinting,

endpoint and parameter discovery, source-map analysis, and OpenAPI parsing.

- Configuration-analysis modules were implemented for TLS, security headers, CORS, authentication cookies, JWT tokens, OpenAPI security anti-patterns, and secret hunting in public GitHub repositories.
- Active probes were implemented for reflected XSS, SQL injection (via sqlmap), IDOR with cross-profile comparison, host header injection, cache poisoning, prototype pollution, open redirect, SSRF / SSTI, XXE, HTTP request smuggling, WebSocket security, race conditions, GraphQL auditing, and OAuth misconfigurations.
- A correlator implementing nine cross-finding rules and an asset-risk module producing a ranked per-host risk score were implemented, allowing the framework to highlight the most exploitable assets at the top of the report.
- A local LLM integration (Ollama with DeepSeek-R1 7B as the default model and an OpenAI-compatible fallback) was implemented and produces an executive-summary report from the structured findings.
- An HTML report renderer, a SIEM-compatible JSON export, a per-session append-only audit log of every HTTP request, and a Telegram notifier for long-running scans were implemented.
- The framework was experimentally evaluated in two complementary settings: a controlled laboratory run against *OWASP Juice Shop* and an authorized real-world case study against the `miras.app` academic management system. The detected vulnerabilities matched the expected categories of both targets, the scope enforcer correctly blocked the single out-of-scope request observed during the real scan, the asset-risk module placed the most exposed administrative subdomain at the top of the ranking, and the full real-target pipeline completed in approximately 1 minute 43 seconds.

Improvements that can be made in future work.

- Add a headless-browser-based DAST stage that drives the application through Playwright in addition to the current HTTP-level probing, in order to discover client-side issues that require an actual JavaScript runtime.

- Extend the active-probe set to cover NoSQL injection, GraphQL batch-based authorization bypass, mass-assignment, and BOLA on REST APIs with finer granularity.
- Introduce a feedback loop in which the LLM proposes follow-up probes (for example, targeted SSRF callbacks against an interactsh server) that are then automatically dispatched by the framework, instead of leaving the proposal to the human analyst.
- Replace the current per-rule correlator with a learned ranker trained on labeled finding sets, in order to improve the prioritization of multi-signal observations.
- Build a dedicated web dashboard — comparable to OWASP DefectDojo — on top of the SIEM-compatible JSON export, with multi-scan diffing and historical trend analysis.
- Package the framework as an installable distribution and add a containerized runtime so that the framework can be deployed both on a local Kali-Linux workstation and inside a CI/CD pipeline.

The work demonstrates that external penetration testing of web applications can be substantially automated without sacrificing explainability, and that local LLMs are mature enough to be integrated as a first-class component of an offensive-security pipeline.

Bibliography

[1] OWASP Foundation.
Owasp top 10: 2021.
https://owasp.org/Top10/, 2021.
Accessed: 2026-05-24.

[2] Karen Scarfone, Murugiah Souppaya, Amanda Cody, and Angela Orebaugh.
Technical guide to information security testing and assessment.
Technical Report NIST Special Publication 800-115, U.S. National Institute of Standards and Technology, 2008.

[3] Penetration Testing Execution Standard project.
The penetration testing execution standard (ptes).
http://www.pentest-standard.org/, 2014.
Accessed: 2026-05-24.

[4] OWASP Foundation.
Owasp web security testing guide (wstg), v4.2.
https://owasp.org/www-project-web-security-testing-guide/, 2020.
Accessed: 2026-05-24.

[5] OWASP Foundation.
Cross site scripting (xss).
https://owasp.org/www-community/attacks/xss/, 2024.
Accessed: 2026-05-24.

[6] PortSwigger Web Security Academy.
What is cross-site scripting (xss) and how to prevent it?
https://portswigger.net/web-security/cross-site-scripting, 2024.
Accessed: 2026-05-24.

[7] hahwul.
Dalfox – powerful open-source xss scanner.
https://github.com/hahwul/dalfox, 2024.
Accessed: 2026-05-24.

[8] OWASP Foundation.
Sql injection.
https://owasp.org/www-community/attacks/SQL_Injection, 2024.
Accessed: 2026-05-24.

[9] Bernardo Damele and Miroslav Stampar.
sqlmap – automatic sql injection and database takeover tool.
https://sqlmap.org/, 2024.
Accessed: 2026-05-24.

[10] OWASP Foundation.
Owasp api security top 10 – api1:2023 broken object level authorization.
https://owasp.org/API-Security/editions/2023/en/
0xa1-broken-object-level-authorization/, 2023.
Accessed: 2026-05-24.

[11] PortSwigger Web Security Academy.
Http host header attacks.
https://portswigger.net/web-security/host-header, 2024.
Accessed: 2026-05-24.

[12] PortSwigger Web Security Academy.
Web cache poisoning.
https://portswigger.net/web-security/web-cache-poisoning, 2024.
Accessed: 2026-05-24.

[13] OWASP Foundation.
Server side request forgery.
https://owasp.org/www-community/attacks/Server_Side_Request_Forgery,
2024.
Accessed: 2026-05-24.

[14] PortSwigger Web Security Academy.
Server-side template injection.

https://portswigger.net/web-security/server-side-template-injection, 2024.
Accessed: 2026-05-24.

[15] OWASP Foundation.
Xml external entity (xxe) processing.
https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing, 2024.
Accessed: 2026-05-24.

[16] OWASP Foundation.
Deserialization cheat sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html, 2024.
Accessed: 2026-05-24.

[17] PortSwigger Web Security Academy.
Http request smuggling.
https://portswigger.net/web-security/request-smuggling, 2024.
Accessed: 2026-05-24.

[18] ProjectDiscovery.
subfinder – fast passive subdomain enumeration tool.
https://github.com/projectdiscovery/subfinder, 2024.
Accessed: 2026-05-24.

[19] OWASP Amass project.
Owasp amass – in-depth attack surface mapping and asset discovery.
https://github.com/owasp-amass/amass, 2024.
Accessed: 2026-05-24.

[20] Gordon “Fyodor” Lyon.
Nmap network scanning.
https://nmap.org/book/, 2009.
Accessed: 2026-05-24.

[21] Wappalyzer.
Wappalyzer technology detection.
https://github.com/AliasIO/wappalyzer, 2024.

Accessed: 2026-05-24.

[22] WPScan Team.
Wpscan – wordpress security scanner.
https://wpscan.com/, 2024.
Accessed: 2026-05-24.

[23] S0md3v.
Arjun – http parameter discovery suite.
https://github.com/s0md3v/Arjun, 2024.
Accessed: 2026-05-24.

[24] ProjectDiscovery.
Nuclei – fast and customizable vulnerability scanner.
https://github.com/projectdiscovery/nuclei, 2024.
Accessed: 2026-05-24.

[25] ProjectDiscovery.
Interactsh – oob interaction gathering server and client library.
https://github.com/projectdiscovery/interactsh, 2024.
Accessed: 2026-05-24.

[26] Ollama project.
Ollama – get up and running with large language models locally.
https://ollama.com/, 2024.
Accessed: 2026-05-24.

[27] DeepSeek-AI.
Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.
https://arxiv.org/abs/2501.12948, 2025.
Accessed: 2026-05-24.

[28] Bjoern Kimminich.
Owasp juice shop.
https://owasp.org/www-project-juice-shop/, 2024.
Accessed: 2026-05-24.

Pipeline and Configuration Reference

This appendix lists the pipeline configuration consumed by the orchestrator at start-up. The full content of `config.yaml.example` is shipped in the repository.

A.1 Pipeline groups

1: Pipeline definition consumed by `main.py`.

```
PIPELINE: list[dict] = [
    {"name": "recon",                 "group": 1},
    {"name": "secret_scanner",        "group": 1},
    {"name": "portscan",              "group": 2},
    {"name": "webdetect",             "group": 2},
    {"name": "techstack",             "group": 3},
    {"name": "spa_crawler",           "group": 3},
    {"name": "fuzzer",                "group": 4},
    {"name": "ssl_checker",           "group": 4},
    {"name": "cors_checker",          "group": 4},
    {"name": "auth_probe",            "group": 4},
    {"name": "cmscan",                "group": 5},
    {"name": "sourcemap_analyzer",    "group": 5},
    {"name": "vhost_enum",            "group": 5},
    {"name": "takeover_checker",      "group": 5},
    {"name": "openapi_parser",        "group": 5},
    {"name": "parameter_discovery",   "group": 6},
    {"name": "http_smuggling",        "group": 7},
    {"name": "jwt_audit",             "group": 7},
    {"name": "websocket_probe",       "group": 7},
    {"name": "api_schema_audit",      "group": 7},
    {"name": "js_security_audit",     "group": 7},
```

```
    {"name": "oauth_probe",            "group": 8},
    {"name": "cache_poison",           "group": 8},
    {"name": "host_header_injection",  "group": 8},
    {"name": "open_redirect_probe",    "group": 8},
    {"name": "prototype_pollution",    "group": 8},
    {"name": "deserialization_probe",  "group": 8},
    {"name": "race_condition",         "group": 8},
    {"name": "graphql_audit",          "group": 8},
    {"name": "api_key_validator",      "group": 9},
    {"name": "xxe_probe",              "group": 9},
    {"name": "injection_probe",        "group": 10},
    {"name": "xss",                    "group": 10},
    {"name": "sql_injection",          "group": 10},
    {"name": "idor_probe",             "group": 10},
    {"name": "vulnscan",               "group": 11},
    {"name": "cve_check",              "group": 12},
    {"name": "correlator",             "group": 13},
    {"name": "asset_risk",             "group": 14},
    {"name": "ai_report",              "group": 15},
]
```

A.2 Excerpt of the configuration schema

2: Excerpt of `config.yaml.example` showing the most important sections.

```
api_keys:
  shodan: ""
  censys_api_id: ""
  censys_api_secret: ""
  github: ""

preset: ""   # safe | bug_bounty | deep | intrusive

auth_profiles:
  anonymous: {}
  user_a:   { token: "", token_type: "Bearer", headers: {}, cookies: {} }
  user_b:   { token: "", token_type: "Bearer", headers: {}, cookies: {} }
  admin:    { token: "", token_type: "Bearer", headers: {}, cookies: {} }

telegram:
  enabled:   true
  bot_token: ""
```

```
  chat_id:   ""
  notify_on: [start, module_complete, critical_finding, complete]

ai:
  enabled:          true
  provider:         "ollama"
  model:            "deepseek-r1:7b"
  ollama_url:       "http://localhost:11434"
  openai_api_key:   ""
  openai_base_url:  "https://api.openai.com/v1"
  temperature:      0.3
  max_tokens:       4096
  language:         "en"
  realtime_advice:  false
  advise_on: [recon, portscan, techstack, fuzzer, vulnscan, cve_check]

scan:
  threads:           50
  timeout:           30
  rate_limit:        50
  global_rate_limit: 50
  allow_write:       false

scope:
  enforce:          true
  allowed_domains:  [localhost]
  allowed_ips:      [127.0.0.1]
  excluded:         []
```

Module Catalogue

This appendix summarizes every module of the framework, with its file, its size in lines of code, and its responsibility. The line counts reflect the snapshot of the repository at the time of writing.

Module	File	LoC	Responsibility
Core			
BaseModule	`modules/base.py`	611	Cross-cutting services: scope, rate limit, audit, subprocess, resume
ActiveProbeBase	`modules/active_probe_base.py`	164	Active-probe mixin: profiles, jitter, dedup, finding builder
FindingRegistry	`modules/finding_registry.py`	752	Unified finding schema and risk scoring
TokenBucket	`modules/rate_limiter.py`	44	Thread-safe rate limiter
AuditLogger	`modules/audit.py`	48	Append-only JSONL audit log
Reconnaissance			
recon	`modules/recon.py`	1038	DNS / subdomain enumeration from 9+ sources
portscan	`modules/portscan.py`	211	Nmap / masscan wrapper
webdetect	`modules/webdetect.py`	366	HTTP/HTTPS probing of resolved hosts
techstack	`modules/techstack.py`	303	Wappalyzer-based technology fingerprinting
spa_crawler	`modules/spa_crawler.py`	202	Headless-browser crawl for SPAs
fuzzer	`modules/fuzzer.py`	1467	Crawl, brute-force, JS secrets, GraphQL detection
cmscan	`modules/cmscan.py`	229	WPScan and CMS-specific scans
vhost_enum	`modules/vhost_enum.py`	121	Virtual-host enumeration
takeover_checker	`modules/takeover_checker.py`	296	Subdomain takeover detection
sourcemap_analyzer	`modules/sourcemap_analyzer.py`	166	.js.map recovery and analysis
openapi_parser	`modules/openapi_parser.py`	202	OpenAPI / Swagger spec discovery and parsing
parameter_discovery	`modules/parameter_discovery.py`	120	Arjun + OpenAPI parameter discovery
Configuration analysis			
ssl_checker	`modules/ssl_checker.py`	258	TLS / certificate / security-header audit
cors_checker	`modules/cors_checker.py`	188	CORS misconfiguration detection
auth_probe	`modules/auth_probe.py`	258	Cookie / Authorization-header audit
jwt_audit	`modules/jwt_audit.py`	313	JWT header / claims / signature audit
oauth_probe	`modules/oauth_probe.py`	310	OAuth endpoint and flow audit

Module	File	LoC	Responsibility
api_schema_audit	modules/api_schema_audit.py	206	OpenAPI security anti-pattern audit
js_security_audit	modules/js_security_audit.py	314	Static analysis of .js files
secret_scanner	modules/secret_scanner.py	298	Secret hunting in public GitHub repositories
api_key_validator	modules/api_key_validator.py	320	Optional live validation of discovered API keys
Active probes			
xss	modules/xss.py	451	Reflected-XSS probing (dalfox + reflection fallback)
sql_injection	modules/sql_injection.py	219	sqlmap wrapper
injection_probe	modules/injection_probe.py	411	SSTI and SSRF probing
idor_probe	modules/idor_probe.py	419	IDOR / BOLA candidate detection and cross-profile comparison
host_header_injection	modules/host_header_injection.py	208	Host / X-Forwarded-Host injection probing
cache_poison	modules/cache_poison.py	238	Web cache poisoning probing
prototype_pollution	modules/prototype_pollution.py	270	Server-side prototype-pollution probing
open_redirect_probe	modules/open_redirect_probe.py	127	Open-redirect detection
deserialization_probe	modules/deserialization_probe.py	158	Insecure-deserialization indicators
xxe_probe	modules/xxe_probe.py	170	XXE indicators with optional OOB
http_smuggling	modules/http_smuggling.py	220	CL.TE / TE.CL / TE.TE desync probing
websocket_probe	modules/websocket_probe.py	300	WebSocket origin / handshake audit
race_condition	modules/race_condition.py	112	Race-condition candidate detection
graphql_audit	modules/graphql_audit.py	142	GraphQL introspection and authorization audit
Aggregation and analytics			
vulnscan	modules/vulnscan.py	369	Nuclei wrapper
cve_check	modules/cve_check.py	228	ExploitDB / CVE correlation
correlator	modules/correlator.py	461	Cross-finding correlation rules
asset_risk	modules/asset_risk.py	291	Per-asset risk score and tier ranking
AI and reporting			
ai_advisor	modules/ai_advisor.py	127	Real-time per-module LLM advice
ai_report	modules/ai_report.py	457	Post-scan LLM analytical report
html_report	reporting/html_report.py	601	Jinja2 HTML report renderer
json_report	reporting/json_report.py	287	SIEM-compatible JSON renderer
telegram	reporting/telegram.py	277	Telegram notification stream

Selected Source Code Listings

This appendix lists three representative source-code excerpts that illustrate the design described in Chapter 2. The complete source code of the framework is shipped in the repository and is not reproduced here in full.

C.1 BaseModule initialization

3: Cross-cutting services attached at module construction (excerpt from `modules/base.py`).

```
class BaseModule:
    name: str = "base"
    description: str = ""
    required_tools: list = []

    def __init__(self, target: str, output_dir: str, config: dict):
        self.target  = target
        self.output_dir = Path(output_dir)
        self.config  = config
        self.domain  = self._clean_domain(target)
        self.rate_limiter = TokenBucket(rate=self._module_rate_limit())
        global_rate = config.get("scan", {}).get("global_rate_limit", 100)
        session_key = str(self.output_dir)
        self._global_limiter = _get_global_bucket(session_key, global_rate)
        self.audit   = AuditLogger(self.output_dir)
        self.results = {}
        self.interrupted_commands: list[str] = []
        self.start_time = None
        self.end_time   = None
        self.module_dir = self.output_dir / self.name
        self.module_dir.mkdir(parents=True, exist_ok=True)
```

C.2 Finding-registry weights and verdict logic

4: Severity / exploitability weights and risk-score helpers (excerpt from `modules/finding_registry.py`).

```
SEVERITY_WEIGHTS = {
    "INFO":      5,
    "LOW":      20,
    "MEDIUM":   45,
    "HIGH":     70,
    "CRITICAL": 90,
}

EXPLOITABILITY_WEIGHTS = {
    "passive":       0.20,
    "candidate":     0.35,
    "requires_auth": 0.50,
    "active":        0.65,
    "confirmed":     0.90,
}

def risk_score(severity: str, confidence: float, exploitability: str) -> float:
    return (
        SEVERITY_WEIGHTS.get(severity, 20)
        * clamp(confidence, default=0.75)
        * _exploitability_weight(exploitability)
    )
```

C.3 AI report invocation

5: Local-LLM provider selection in the post-scan analysis module (excerpt from `modules/ai_report.py`).

```
def run(self) -> dict:
    ai_cfg = self.config.get("ai", {})

    if not ai_cfg.get("enabled", True):
        self.info("AI analysis disabled in config")
        return {"analysis": "", "status": "disabled"}

    provider = ai_cfg.get("provider", "ollama")
    model    = ai_cfg.get("model", "deepseek-r1:7b")
    lang     = self._normalize_language(ai_cfg.get("language", "en"))
```

```
prompt = self._build_prompt(lang)
self.save_text(prompt, "ai_prompt.txt")
self.info(f"Provider: {provider}  Model: {model}")

if provider == "ollama":
    ollama_url = ai_cfg.get("ollama_url", "http://localhost:11434")
    if not self._ollama_alive(ollama_url):
        self.warn("Ollama not running -- skipping AI analysis")
        return {"analysis": "", "status": "ollama_unavailable"}
    analysis = self._ollama_generate(ollama_url, model, prompt, ai_cfg)
else:
    analysis = self._openai_compatible_generate(prompt, ai_cfg)
# ... language validation and return ...
```